Sadržaj

Uvod 1

1. Relevantne pozadinske informacije i literatura 2

1.1 Osnove o eroziji 2

1.2 Postojeća rješenja 2

2. Model erozija tla pod utjecajem kapljica 4

2.1 Fizika 5

2.1.1 Računanje smjera kretanja 5

2.1.2 Erozija tla 7

2.1.3 Dodatna svojstva čestice 8

3. Radna okolina 8

3.1 Unity 8

3.2 Skripte 8

4. Programska izvedba modela erozije tla 12

4.1 Generiranje terena 12

4.1.1 Perlin noise 13

4.1.2 Ostvarivanje boje 14

4.2 Programiranje fizike 15

4.3 Parametri 17

4.3.1 Postavke generacije terena 17

4.3.2 Postavke erozije 24

4.4 Korisničko sučelje 34

5. Rezultati 35

Zaključak 39

Literatura 40

Sažetak.....41

Summary.....42

Skraćenice.....43

Uvod

Svijet oko nas je oblikovan s izrazitom velikom količinom faktora, prirodnih i ljudskih. Sve nizine i visine oko nas stvorene su kroz godine i godine promjena. Jedan od glavnih faktora i čimbenika tih promjena jest upravo voda.

Erozija uzrokovana vodom, tj. hidraulička erozija, je proces uklanjanja određene količine zemlje i kamena, te njihovo akumuliranje u neku drugu točku na reljefu. Taj proces je najčešće uzrokovan kišom kroz godine atmosferskih događanja. Snaga erozije ovisi o kriterijima padine, vrste tla, kapacitetu čestica, intenzitetu kiše i mnogima drugima. [1]

Kao cilj ovog završnoga rada, želimo simulirati upravo taj izolirani čimbenik erozije i gledati kako on utječe na neki unaprijed definiran teren. Uz ovo trebali bi ostvariti i jednostavne mogućnosti vlastitog konfiguriranja osnovnih parametara koji će biti opisani kasnije.

Simulacije poput ovih su bitne zbog jednostavnog razloga da je modeliranje realističnih terena (pogotovo na većoj razini) izrazito mukotrpan i dugotrajan proces. Kada bi imali alat koji automatski generira realistične rezultate, samo vrijeme koje bi uložili bi se podosta smanjilo.

Više je metoda za ostvarivanje simulacija ove teme, no u ovome radu ćemo se osvrnuti na korištenje visinske mape (eng. *heightmap*). S definiranim skupom podataka nad kojim ćemo raditi, preostalo nam je stvoriti jednostavan algoritam koji će za svaku česticu definirati putanju, količinu sedimenta koje će nositi i koje će ostaviti putem.

Najprije ćemo definirati teoriju, fiziku i osnovne uvjete procesa erozije, te kako se tome problemu pristupalo kroz povjest i upoznati se sa relevantnom znanstvenom literaturom. Tek onda se možemo posvetiti modelu i samom programskom rješenju kako je računalno ostvaren te kako se s njime baratati.

1. Relevantne pozadinske informacije i literatura

1.1 Osnove o eroziji

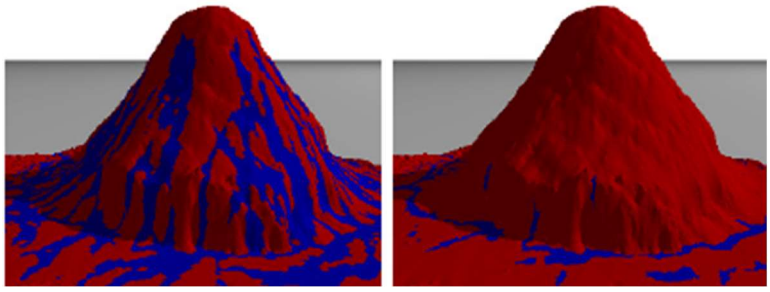
Već smo ustanovili da je erozija proces koji direktno utječe na izgled terena. Trebamo uspostaviti skup pravila kojima se taj proces vodi kako bi ta pravila mogli prevesti u računalni program. U korist toga, fokusirat ćemo se najprije na najmanji element – kapljicu.

Životni tijek jedne kapi vode u stvarnome svijetu je generalno teško za pratiti. To je naravno zbog činjenice da se voda ubrzo spaja s ostalim kapima vode te kreiraju veća tijela. No da bi lakše vizualizirali problem, pokušat ćemo zamisliti da je kap vode uvijek konstantan element, poput jedne loptice od njih tisuće. Kada se kapljica stvori u atmosferi, ona pod utjecajem gravitacije pada na tlo, dajući joj neku početnu brzinu. Pri kontaktu s tlom kreće u smjeru lokalnog minimuma. Njezin životni vijek traje sve dok ne ispari, te hoće li se pronaći u lokalnom minimumu ili ne ovisi o poziciji u kojoj se stvorila.

Tu nailazimo na prvi logički problem ovoga rada – što ako kap vode upadne u lokalni minimum odmah? U tom slučaju, svijet bi uvijek izgledao kao skupina jama. Dakle, svakoj kapljici moramo dati neki mali pogon, to jest silu koja bi ju izvukla iz nekog manjeg minimuma. To ostvarujemo inercijom.

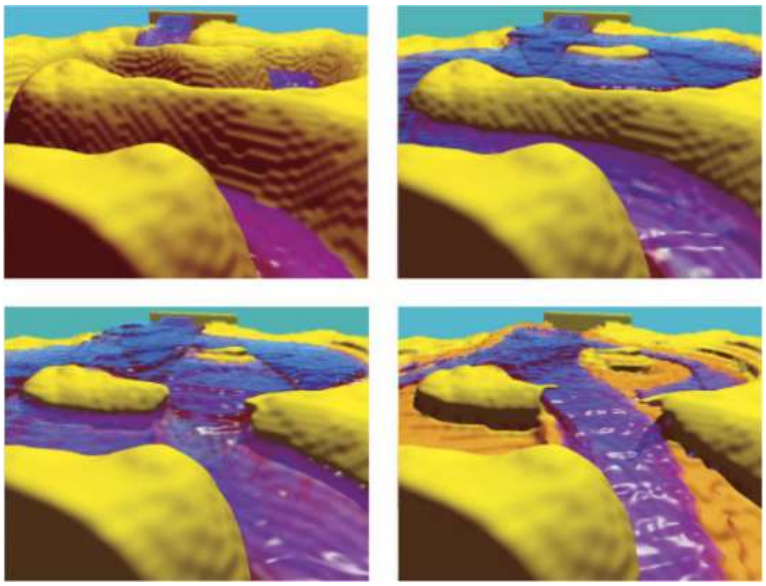
1.2 Postojeća rješenja

Pregled literature nas upućuje u razne prijašnje radove koji su već definirali danu temu i njezina rješenja. Kao jedno od prvih akademskih radova koji se bave izričito vizualnom simulacijom erozije terena uzrokovane vodom bio bi rad B. Baneša 2002. godine koji uvodi algoritam erozije kao proces s četiri neovisna koraka. Model ima mogućnost erodiranja terena, transportiranja terena i taloženja na drugom mjestu, te uz to metode isparavanja i zasićenja kapljica vode. Model je bio revolucionaran u smislu da je prvi, na neku ruku, "grafički" model koji se upravo bavi temom erozije [2].



Sl. 1.1 Prikaz erozije terena Marsa nakon kiše u radu B. Baneša [2]

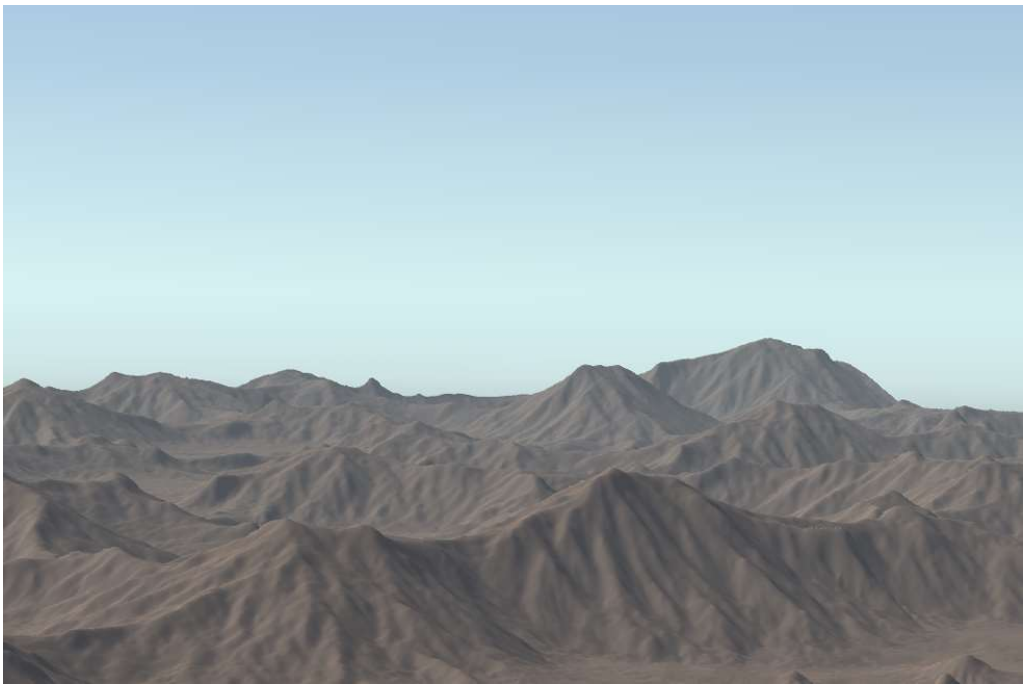
Inspiriran idejom iz prijašnjeg rada, B. Baneš ponovo stvara novi model erozije, implementacijom visinske mape. Ta ideja je bila izrazito dobra u smislu da se svi podaci mogu lakše računati kada se teren može prikazati kao jednostavan koordinatni sustav, te prikazati u 2D slici. [3]



Sl. 1.2 Erozija rijeke na teren u radu B. Baneša [3]

Rad koji je postavio najbolje temelje ovome završnome radu napisao je Hans Theobald Beyer. Rad ima dobru akumulaciju prijašnjih izvora s fokusom na eroziju terena definiranog kao visinska mapa. Upravo u tome radu je opisan model koji pokušavamo replicirati i nadograditi u svrhe ovoga rada. Opisane su formule i pravila koje čestice moraju poštivati kako bi rezultat erozije bio ne samo realističan, već i brz. [4]

Implementacija metode hidrauličke erozije ime je rada Hansa T. Beyera u kojem se akumulira i analizira skup radova kroz prošlost koji su se posvetili ovoj temi, te na temelju istih stvara matematičke i fizikalne formule s kojima je postignut i ovaj završni rad.



Sl. 1.3 Erodirani teren H. Bayera [4]

Slika 1.3 koja prikazuje erodirani teren u radu H. Bayera služi kao nekakav cilj koji bi htjeli postići, nešto što bi bilo moguće i implementacijom algoritma u ovome radu.

2. Model erozija tla pod utjecajem kapljica

Povijest simulacija erozije proteže se od početka računalne grafike. Potreba za brzim algoritmom za stvaranje realističnih rezultata u obliku terena je stalna, te su kroz godine znanstvenici i programeri radili zajedno kako bi mogli postići taj cilj.

S tim ciljem na umu, orijentiramo se prema završnome radu Hansa Theobalda Beyera s Tehničkog sveučilišta u Minhenu [4]. Osvrnuti ćemo se na rad Hansa T. Beyera i priložit ćemo naše rješenje koje je nastalo na temelju njihovog, potkrjepljujući s matematičkim formulama za bolje razumijevanje. Bitno je naglasiti da model opisan u našem završnome radu ne prati kompletno definiran model u radu H. Bayera, zbog jednostavnog razloga da je fokus ovoga

završnoga rada bio u ostvarivanju bržih i efikasnijih rezultata, dakle algoritam koji bi se mogao provoditi konstantno, a da naše računalo previše instrukcija mora provoditi.

Svaka čestica vode sadrži sljedeće informacije: poziciju na terenu, smjer kretanja, brzinu kretanja, količinu vode koju sadrži. S ovim varijablama, definiraju se formule kojima računamo kretnju i utjecaj čestica na teren.

2.1 Fizika

2.1.1 Računanje smjera kretanja

Gradijent, to jest smjer kretanja, određuje u kojem smjeru vektor brzine utječe na česticu.

U povezanome radu, jasno su definirana pravila pokreta. Koristi se metoda dvolinearne interpolacije. Dvolinearna interpolacija je metoda računanja neke vrijednosti uzimajući u obzir četiri susjedne točke u prostoru [5]. U primjeni rada, računa se pomoću četiri susjedna gradijenta točaka polja. Takav pristup osigurava preciznost svim područjima polja. Pomoću trenutačne pozicije točke $pos_{old} = (x + u, y + v);\quad u, v \in [0,1]$, gradijenti susjednih točaka $P_{x,y}$, $P_{x+1,y}$, $P_{x,y+1}$, $P_{x+1,y+1}$ definirani su sa:

$$g(P_{x,y}) = \begin{pmatrix} P_{\mathrm{x+1,y}} - P_{x,y} \\ P_{\mathrm{x,y+1}} - P_{x,y} \end{pmatrix}$$

$$g(P_{x+1,y}) = \begin{pmatrix} P_{\mathrm{x+1,y}} - P_{x,y} \\ P_{\mathrm{x+1,y+1}} - P_{x+1,y} \end{pmatrix}$$

$$g(P_{x,y+1}) = \begin{pmatrix} P_{\mathrm{x+1,y+1}} - P_{x,y+1} \\ P_{\mathrm{x,y+1}} - P_{x,y} \end{pmatrix}$$

$$g(P_{x+1,y+1}) = \begin{pmatrix} P_{\mathrm{x+1,y+1}} - P_{x,y+1} \\ P_{\mathrm{x+1,y+1}} - P_{x+1,y} \end{pmatrix}$$

Provođenjem dvolinearne interpolacije, dolazi se do konkretne definicije gradijenta g u točki pos_{old}:

$$g(pos_{old}) = \begin{pmatrix} (P_{\mathrm{x+1,y}} - P_{x,y}) * (1 - v) + (P_{\mathrm{x+1,y+1}} - P_{x,y+1}) * v \\ (P_{\mathrm{x,y+1}} - P_{x,y}) * (1 - u) + (P_{\mathrm{x+1,y+1}} - P_{x+1,y}) * u \end{pmatrix}$$

Te s time toga se dolazi do konkretne formule za smjer kretanja:

$$dir_{new} = dir_{new} * p_{inertia} - g * (1 - p_{inertia}) \qquad pos_{new} = pos_{old} + dir_{new}$$

Ova metoda je kvalitetna i izuzetno precizna, no za korist stvaranja simulacije koja će u stvarnom vremenu morati prikazati nekakve rezultate, moramo koristiti nekakvu metodu koja bi davala zadovoljavajuće rezultate u izuzetno brzom vremenu. Za dobivanje toga, koristit ćemo se diskretnom procjenom baziranom na centralnoj razlici. Centralna razlika je metoda numeričke procjene korištene za računanje razliku vrijednosti funkcije u susjednim točkama, konkretno ovo je zapravo aproksimacija derivacije [6]. Drastično pojednostavljeno računanje za dobivanje aproksimativnog rješenja ovdje nam uštađuje računalne resurse u korist boljeg bržeg prikaza. Funkcionira tako da definiramo česticu na točki s koordinatama x, odnosno y, te visinu $h = H(x, y)$. Još odredimo četiri susjeda oko točke, lijevo $H(x-1, y)$, desno $H(x+1, y)$, gore $H(x, y+1)$ i dolje $H(x, y-1)$.

Sada definiramo gradijent:

$$g_{x,y} = \begin{pmatrix} (H_{x+1,y} - H_{x,y}) - (H_{x-1,y} - H_{x,y}) \\ (H_{x,y+1} - H_{x,y}) - (H_{x,y-1} - H_{x,y}) \end{pmatrix}$$

, te provodimo normalizaciju gradijenta:

$$|\vec{g}| = \sqrt{{g_x}^2 + {g_y}^2}$$

Na kraju imamo formulu za konačni gradijent:

$$\hat{g} = \frac{\vec{g}}{|\vec{g}| + \varepsilon}; \qquad \varepsilon = 10^{-6}$$

, gdje dodajemo faktor ε kako bi izbjegli dijeljenje s nulom.

Kada smo izračunali novi gradijent, možemo preusmjeriti česticu. Odmah definiramo i jednu konstantu, inerciju, koja će diktirati pokret čestice kroz sve njezine korake. Primjenjujmo istu formulu definiranu u citiranom radu:

$$dir_{new} = \begin{pmatrix} dir_x = dir_x * p_{inertia} - g * (1 - p_{inertia}) \\ dir_y = dir_y * p_{inertia} - g * (1 - p_{inertia}) \end{pmatrix}$$

, te dodatno provedemo normalizaciju.

$$\widehat{dir} = \frac{\overrightarrow{dir}}{|\overrightarrow{dir}|}$$

Na kraju sve što je preostalo je izračunati novu poziciju jednostavnim dodavanjem.

$$pos_{new} = pos_{old} + dir_{new}$$

2.1.2 Erozija tla

Za pristupiti samom problemu erozije tla, gledamo svaku česticu vode kao mali transportni kamion. Kapacitet kamiona izražen je parametrom *capacity*. Pristupamo problemu na slični način kao u usporednom radu, no u svrhu eksperimentiranja ostavljamo otvorene varijable *sedimentCapacityFactor* i *minSedimentCapacity*. S time na umu, proučavamo kalkulaciju kapaciteta kao jednostavan izračun maksimuma umnoškom svih faktora:

$$c = max(-\Delta h \cdot v \cdot w \cdot sedimentCapacityFactor, minSedimentCapacity)$$

, gdje je Δh promjena visina između stare i nove točke, v brzina čestice i w količina vode u čestici.

Erozija je zapravo uvjet koji konstantno provjeravamo dok čestica putuje po terenu. U usporednom radu erozija se distribuira podjednako po svim susjednim točkama u određenom radijusu oko čestice. Zbog jednostavnosti ovaj proces se može smanjiti na fundamentalnu razinu tako da erozija se dešava samo u jednoj točki. Rezultati će biti zadovoljavajući i uštedjeti ćemo puno računalnih resursa. Dakle definiramo uvjet koji se konstantno provjerava i ako je ispunjen ostvaruje proces erozije.

$$sediment < c \ \rightarrow \ eroded = min((c - sediment) \cdot erodeSpeed, map[idx])$$

, gdje je *sediment* količina tla koju čestica nosi, a *eroded* količina tla koju čestica erodira. U točki *map[idx]* se oduzme od visine erodirana količina.

Svo tlo iskopano erozijom se mora negdje i ostaviti. Zato na svakoj čestici imamo uvjet, ako je količina tla koja čestica sadrži veća od kapaciteta njezinoga, ona ostavlja iza sebe višak tla, ili formulama:

$$sediment > c \ \rightarrow \ deposited = (sediment - c) \cdot depositSpeed$$

, gdje je *depositSpeed* faktor koji utječe na brzinu ostavljanja.

2.1.3 Dodatna svojstva čestice

Svaka čestica ima i interna svojstva koja određuju i potiču njezinu aktivnost. Ta svojstva su brzina i količina vode, te ih računamo na jednak način kao u usporednom radu.

Čestice su stvorene s nekom brzinom kad se stvore, ta varijabla je dostupna korisniku da promjeni po volji. Naravno, to sve spada pod pretpostavku da su čestice poput kapi kiše, te dobiju određenu brzinu u startu padom iz atmosfere. Kada se ta čestica kreće po terenu, konstantno ažuriramo njezinu brzinu kako bi dobili realistični pokret. To radimo s formulom:

$$v_{new} = \sqrt{{v_{old}}^2 + \Delta h * gravity}$$

, gdje je Δh promjena visine između stare i nove točke, a *gravity* otvorena varijabla gravitacije.

Uz faktor brzine, moramo i dati određenu količinu snage toj vodi. To ćemo spremati u varijabli *water* koja će polagano isparavati kako prolazi vrijeme. Opisano formulom:

$$w_{new} = w_{old} * (1 - evaporateSpeed)$$

, gdje je *evaporateSpeed* ∈ [0,1] otvorena varijabla koja opisuje koliko voda brzo isparava.

3. Radna okolina

3.1 Unity

Unity je višeplatformski pogon za videoigre i slične 3D aplikacije, kojeg aktivno razvija Unity Technologies [7]. Koristi se najčešće za videoigre, no može se koristiti i za simulacije kao što je slučaj u ovome radu. Zbog jednostavnosti i mogućnosti stvaranja skripti i algoritama, izabran je kao podloga za ovu simulaciju.

3.2 Skripte

U kontekstu Unity-a, skripte su vrsta komponenti koje utječu da ponašanje objekata u Unity-u. Pisane su koristeći objektno orijentiran programski jezik C#, te se automatski pokreću kada je aplikacija u aktivnom stanju.

Srž simulacije su same skripte, svaka sa svojom funkcionalnosti. Obuhvatit ćemo sve skripte kako bi se upoznali sa strukturom našega rada te ćemo onda detaljnije o najbitnijim komponentama, uključujući i programski kod, u poglavlju Programska izvedba.

Simulacija je sadržana od sedam skripta, s glavnom podjelom na kontrolere i generatore, te jednom skriptom za kretnju kamere.

CameraRotator

CameraRotator.cs je skripta koja nam omogućava kretnju kamere oko terena. S njom možemo proučiti model i detaljnije obuhvatiti sve njegove strane. Definirane su tri metode za pokret kamere, vučenje miša lijevim klikom vrti kameru oko terena, vučenje s pritisnutim desnim klikom okreće kameru oko svoje osi. Pokretanjem kotačića miša gore, odnosno dolje zumira prema, odnosno dalje od terena. Konačna pozicija kamere je izračunata konverzijom u Kartezijev sustav kako bi kameru pozicionirali na kružnoj putanji oko terena.

ErosionController

ErosionControler je glavna skripta za računanje erozije, te je u njoj sadržana kompletna logika za kretnju čestica. Kroz nju provodimo podatke o visinskoj mapi te ih obrađujemo stvaranjem određene količine čestica. Sadrži sve parametre vezane za eroziju koji će detaljnije biti opisani u djelu Parametri. Metode koje prevladavaju u ovom kontroleru su Erode() i HydraulicErosion(). Erode() metoda je funkcija koja dohvaća teren te ga konvertira u visinsku mapu. Nakon toga, poziva HydraulicErosion() koji izvršava eroziju i mijenja tu visinsku mapu u novu, erodiranu. Na kraju metoda stvara mrežu od te visinske mape.

Metoda HydralicErosion() stvara eroziju. U ovome poglavlju ćemo se dotaknuti logike i priložiti pseudokod funkcije, a u poglavlju Programska izvedba ćemo detaljnije pogledati konkretne linije iz koda i usporediti ih s prije spomenutim funkcijama. Dakle, ova metoda je glavna za simuliranje erozije. Najprije se nasumično izabere mjesto na terenu i postavi kap te joj se daju početne definirane vrijednosti. Onda pomoću gore navedenih pravila, računamo gradijent, pomicanje sedimenta i gubitak nekih vrijednosti koje se sadrži u kapi, poput količine vode. Ovaj proces ponavljamo onoliko iteracija koliko smo definirali. U pseudokodu, ta implementacija izgleda ovako:

```
za i u iterations:
    nasumično izaberi poziciju (posX, posY)
    za svaki korak do maxLifetime:
        izračunaj gradijent visine (gradX, gradY)
        normaliziraj gradijent
        promjeni smjer kretanja:
            izracunaj dir
            normaliziraj dir
        pomakni kapljicu:
            pos += dir
        ako kapljica izađe izvan granica:
                  break
        izračunaj novu visinu i promjenu visine deltaH
        izračunaj kapacitet
        ako kapacitet < sediment:
            izracunaj deposit
            sediment += deposit
        inače:
            izracunaj kolicinu erozije erodeAmount
            visina -= erodeAmount
            sediment += erodeAmount
        ažuriraj brzinu
        ispari vodu
        ako nema dovoljno vode:
                  break
```

ErosionOverTimeController

ErosionOverTime je jednostavna skripta koja samo iterativno poziva algoritam za eroziju kako bi dala dojam animacije, to jest eroziju kroz nekakav duži period vremena. Prilikom inicijalizacije, skripta dohvata referencu na ErosionControler skriptu, te s StartErosion() metodom započinje rutinu ErodeOverTime(), koja zapravo sve što radi je zove Erode() iz prethodno opisane skripte, s kratkom pauzom između svakog poziva. Frekvencija koraka određena je s javnom varijablom erosionStepsPerSecond, te samo trajanje rutine je defnirano s totalRuns varijablom. Postoji mogućnost ranijeg zaustavljanja rutine s metodom StopErosion().

S ovom skriptom zapravo ostvarujemo poantu završnoga rada – prikaz erozije u nekom ljudsko-primjetnom vremenu.

HeightMapGenerator

Skripta HeightMapGenerator je komponenta odgovorna za proceduralno generiranje terena u obliku visinske mape. Koristi slojeviti Perlinov šum da bi simuliralo nasumičnost terena i daje realistične rezultate. Perlinov šum (*eng. Perlin noise*) vrsta je vizualne buke koja nastaje spajanjem slojeva transparentne slike kako bi dobila konačna slika, nalik statičkoj buci koju televizor daje kada nema signala [8]. Metoda generateHeightMap() stvara teren veličine $mapsize \times mapsize$. Nasumično stvara svaku oktavu vizualne buke, te za svaku oktavu

postavljamo drugačiji pomak, povećanje frekvencije i postojanost. Na kraju spajamo sve te slojeve dodavanjem adekvatno jačine svake iste koordinate. Ovo nam omogućuje fraktalne rezultate i lijepu bazu za erodiranje. Na kraju provodimo normalizaciju da bi standardizirali kontrast terena i izbjegli nekonzistentnost.

```
za svaki x,y u mapSize:
      noiseValue = 0, scale = initialScale, weight = 1
      za svaki oktav i u numOctaves:
            p = offset[i] + (x / mapSize, y / mapSize) * scale
            PerlinNoise(p.x, p.y) * weight
            noiseValue += PerlinNoise
            weight = weight * persistence
            scale = scale * lacunarity
      spremi noiseValue u map[y * mapSize + x]
      ažuriraj minValue i maxValue
ako maxValue ≠ minValue:
  za svaki indeks i u map:
    normaliziraj map[i]
vrati map
```

TerrainMeshGenerator

TerrainMeshGenerator je skripta koja generira i ostvaruje 3D prikaz mreže terena na temelju visinske mape. Mreža se stvara izračunom položaja vrhova te indeksira trokute, te zatim postavlja materijal za prikaz.

Pošto moramo dinamički generirati mrežu na temelju novo izračunate visinske mape, imamo mogućnost generiranja mreže inicijalno te dinamički kako se erozija provodi. Metoda GenerateMesh() uzima visinsku mapu iz HeightMapGenerator komponente i stvara mrežu s tom mapom. Metoda GenereteMeshFromHeightmap() uzima postojeću novu visinsku mapu (koja nastaje na temelju stare, dakle nakon erozije), te od nje stvara mrežu trokuta. S ovime postižemo konstanti prikaz terena.

Skripte za UI

Uz sve skripte za računanje erozije, moramo imati nekakav korisnički unos kako bi model mogao primati nekakve informacije od korisnika i prikazati ih adekvatno. U tu korist stvaramo korisničko sučelje. Skripte koje su odgovorne za korisničko sučelje su ErosionUIController i WorldGenUIController.

ErosionUIController sadrži sve relevantne parametre vezane za proces erozije, te omogućuje korisniku da barata s njima kako želi. Može mijenjati broj iteracija, inerciju, kapacitet čestica za sediment i sve ostale parametre, koji će više biti opisani u poglavlju Parametri.

WorldGenUIController je skripta koja upravlja sučeljem za generiranje terena. Omogućuje korisniku unos parametara za generiranje terena poput veličine terena, visine, postavke generiranja, te mu omogućuje sve to uz klik gumba.

Uz navedene skripte imamo još dodatna dva gumba za eroziju, jedan za prikaz samo jednog koraka erozije, te drugi za prikaz erozije u vremenu.

4. Programska izvedba modela erozije tla

Sada kada smo definirali model koji ćemo koristiti i sve skripte te kako bi trebale raditi, možemo se posvetiti više programskome djelu ovoga rada te prikazati kako smo računalno ostvarili ono što smo do sada definirali.

4.1 Generiranje terena

Proces generiranja terena sastoji se od dva glavna koraka, generiranja visinske mape koje je ostvareno u skripti HeightMapGenerator, te generiranja mreže trokuta u TerrainMeshGenerator. U HeightMapGeneratoru koristimo Perlinov šum kako bi postigli generiranje nasumičnog terena u obliku dvodimenzionalnog niza s vrijednosti visina. Detaljnije o Perlinovom šumu i kako funkcionira ćemo navesti u poglavlju Perlin noise. U prikazanoj rekurzivnoj funkciji prolazimo kroz svaku poziciju u visinskoj mapi te joj definiramo njezinu visinu.

```
for (int y = 0; y < mapSize; y++) {
         for (int x = 0; x < mapSize; x++) {
                float noiseValue = 0;
                float scale = initialScale;
                float weight = 1;
                for (int i = 0; i < numOctaves; i++) {
                    Vector2 p = offsets[i] + new Vector2(x /
(float)mapSize, y / (float)mapSize) * scale;
                    noiseValue += Mathf.PerlinNoise(p.x, p.y) *
weight;
                    weight *= persistence;
                    scale *= lacunarity;
                }
                map[y * mapSize + x] = noiseValue;
                minValue = Mathf.Min(noiseValue, minValue);
                maxValue = Mathf.Max(noiseValue, maxValue);
            }}
```

Sada kada smo vratili mapu moramo od nje napraviti mrežu. U TerrainMeshGenerator se nalaze dvije funkcije za stvaranje te mreže, no obje prate isti princip. Uzimamo visinsku mapu za

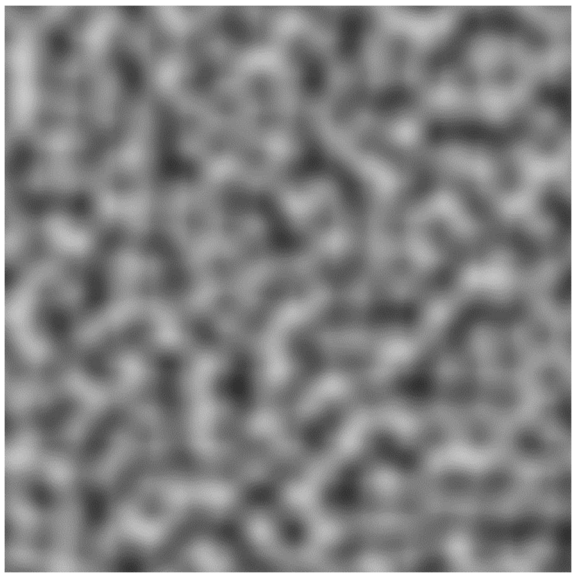
izgradnju trodimenzionalne mreže terena, gdje svaka točka mreže dobiva svoju visinu prema vrijednosti iz visinske mape.

```
for (int y = 0; y < mapSize; y++)
        {
            for (int x = 0; x < mapSize; x++)
            {
                int i = y * mapSize + x;
                float height = heightMap[i] * meshHeight;
                vertices[i] = new Vector3(x, height, y);
                uvs[i] = new Vector2(x / (float)(mapSize - 1),
y / (float)(mapSize - 1));
            }
        }
        int t = 0;
        for (int y = 0; y < mapSize - 1; y++)
        {
            for (int x = 0; x < mapSize - 1; x++)
            {
                int i = y * mapSize + x;
                triangles[t++] = i;
                triangles[t++] = i + mapSize;
                triangles[t++] = i + 1;
                triangles[t++] = i + 1;
                triangles[t++] = i + mapSize;
                triangles[t++] = i + mapSize + 1;
            }
        }
```

Ovo su glavne petlje za računanje svih trokuta. Prva petlja prolazi kroz sve točke mreže i za svaku točku računa visinu na temelju visinske mape i parametra meshHeight, te postavlja poziciju vrha u 3D prostoru i dodjeljuje koordinate za teksture. Druga petlja generira trokute koji definiraju površinu mreže. Svaki se kvadrat na mreži dijeli na dva trokuta kako bi se cijeli teren mogao prikazati uz pomoć trokuta.

4.1.1 Perlin noise

Kao što smo već ustanovili, Perlinov šum je algoritam koji kreira buku u korist stvaranja vizualne reprezentacije nasumičnosti. Razvio ga je Ken Perlin 1983. godina kada je imao potrebu za stvaranje tekstura i vizualnih efekata za film Tron 1982. Zanimljivo da je za svoje otkriće dobio Oskar za tehnološko postignuće, upravo zbog korisnosti i izrazito široke primjene [8]. Iako je razvijen za vizualne efekte u filmu, korisnost Perlinovog šuma je daleko nadmašila njegova očekivanja, pogotovo u brzo razvijajućem svijetu računalne grafike. Kada želimo generirati nekakav nasumičan teren, koristimo upravo nasumičan generator te je Perlinov šum postao temelj za sve kasnije implementacije nasumično generiranog terena, kao i ovog opisanog u našem radu.



Sl. 4.1 Prikaz Perlinovog šuma, preuzeto s Wikipedije

4.1.2 Ostvarivanje boje

Boju u našem radu smo postigli korištenjem sjenčara, to jest *eng. shader.* Shader je program koji se izvršava na grafičkoj kartici i određuje kako će se površina terena prikazati na ekranu. Specifično za Unity, shader definira kako se boje, svijetlo i tekstura spajaju u jednu specifičnu boju za svaki piksel na ekranu.

TerrainSurface.shader je naša implementacija ovoga, to je shader koji omogućuje dinamičko bojanje površine na temelju njezine visine i nagiba. Koristimo četiri vrste boje, trava (zeleno), zemlja (smeđe), stijena (sivo) i snijeg (bijelo). S ovime postižemo prividni rezultat realistične površine. Shader prima podatke o svakoj točki površine te računa visinu i nagib te točke. Na osnovi toga adekvatno postavlja boju piksela.

```
void surf (Input IN, inout SurfaceOutputStandard o)
        {
            float height = IN.worldPos.y;
            float slope = 1.0 - IN.worldNormal.y;

            float snowMask = smoothstep(_SnowHeight -
_HeightBlend, _SnowHeight + _HeightBlend, height);
            float rockMask = smoothstep(_RockSlope -
_SlopeBlend, _RockSlope + _SlopeBlend, slope);
            float dirtMask = smoothstep(_GrassSlope -
_SlopeBlend, _GrassSlope + _SlopeBlend, slope) * (1.0 -
rockMask);
            float grassMask = 1.0 - dirtMask - rockMask -
snowMask;
```

```
            fixed4 c = _GrassColor * grassMask + _DirtColor *
dirtMask + _RockColor * rockMask + _SnowColor * snowMask;
            o.Albedo = c.rgb;
            o.Alpha = 1;
        }
```

4.2 Programiranje fizike

Maloprije smo definirali funkcije i pseudokod za ostvarivanje erozije terena. S time možemo se posvetiti programskom rješenju toga i kako računamo eroziju. Segmentirat ćemo metodu za eroziju, HydraulicErosion, te provjeriti kako s njom dolazimo do konačnog rezultata.

Erozija se pokreće u glavnoj metodi Erode(). U njoj provjeravamo postoji li teren i ako postoji postavljamo visinsku mapu i šaljemo sve podatke u HydraulicErosion() za izračun erozije.

Inicijalizacija

```
void HydraulicErosion(float[] map, int mapSize, int iterations)
{
    System.Random prng = new System.Random();
    int[] dx = {-1, 1, 0, 0};
    int[] dy = {0, 0, -1, 1};
    for (int iter = 0; iter < iterations; iter++) {
        // randomly place a droplet
        float posX = prng.Next(erosionRadius, mapSize -
erosionRadius);
        float posY = prng.Next(erosionRadius, mapSize -
erosionRadius);
        float dirX = 0, dirY = 0;
        float speed = startSpeed;
        float water = startWater;
        float sediment = 0;
        for (int lifetime = 0; lifetime < maxLifetime;
lifetime++) {
            ...
        }
    }
}
```

Najprije za svaku kapljicu nasumično biramo početnu poziciju na mapi. Svaka kap ima svoju brzinu, količinu vode i kapacitet sedimenta. Također definiramo životni vijek čestice koji se konstantno smanjuje, poput Time to Live u smislu paketa u računalnim mrežama.

Pomak

```
int nodeX = (int)posX;
int nodeY = (int)posY;
int idx = nodeY * mapSize + nodeX;
float gradX = 0, gradY = 0;
float h = map[idx];
for (int d = 0; d < 4; d++) {
    int nx = nodeX + dx[d];
    int ny = nodeY + dy[d];
```

```
        if (nx >= 0 && nx < mapSize && ny >= 0 && ny < mapSize) {
            float nh = map[ny * mapSize + nx];
            gradX += (nh - h) * dx[d];
            gradY += (nh - h) * dy[d];
        }
    }
    float gradMag = Mathf.Sqrt(gradX * gradX + gradY * gradY) + 1e-
    6f;
    gradX /= gradMag;
    gradY /= gradMag;
    dirX = dirX * inertia - gradX * (1 - inertia);
    dirY = dirY * inertia - gradY * (1 - inertia);
    float len = Mathf.Sqrt(dirX * dirX + dirY * dirY);
    if (len == 0) break;
    dirX /= len;
    dirY /= len;
    posX += dirX;
    posY += dirY;
```

Nadalje, bavimo se računanjem gradijenta terena u trenutačnoj lokaciji. S time ćemo izračunati smjer kapljice, uzimajući u obzir inerciju, smjer kretanja i gradijent. S time, pomičemo kapljicu u tom smjeru.

Primijetimo računalnu implementaciju svih prije opisanih formula:

$$g_{x,y} = \begin{pmatrix} (H_{x+1,y} - H_{x,y}) - (H_{x-1,y} - H_{x,y}) \\ (H_{x,y+1} - H_{x,y}) - (H_{x,y-1} - H_{x,y}) \end{pmatrix}$$

$$dir_{new} = \begin{pmatrix} dir_x = dir_x * p_{inertia} - g * (1 - p_{inertia}) \\ dir_y = dir_y * p_{inertia} - g * (1 - p_{inertia}) \end{pmatrix}$$

$$pos_{new} = pos_{ol} \ + dir_{new}$$

Erozija i taloženje

```
int newIdx = ((int)posY) * mapSize + (int)posX;
float newH = map[newIdx];
float deltaH = newH - h;
float capacity = Mathf.Max(-deltaH * speed * water *
sedimentCapacityFactor, minSedimentCapacity);
if (sediment > capacity) {
    float deposit = (sediment - capacity) * depositSpeed;
    map[idx] += deposit;
    sediment -= deposit;
} else {
    float erodeAmount = Mathf.Min((capacity - sediment) *
erodeSpeed, map[idx]);
    map[idx] -= erodeAmount;
    sediment += erodeAmount;
}
```

Konstantno kapljica malo uzima i malo daje od terena kojim prolazi. U tu svrhu imamo nekolicinu provjera s kojima računamo kako će se kap ponašati prema terenu. Ako kapljica nosi

više sedimenta nego što može, dio sedimenta se taloži na trenutačnoj poziciji. Ako može ponijeti više, erodira teren i pokupi dodatni sediment.

Ponovo možemo primijetiti jasne linije koje reprezentiraju prije objašnjene formule:

$$c = max(-\Delta h \cdot v \cdot w \cdot sedimentCapacityFactor, minSedimentCapacity)$$

$$sediment < c \ \rightarrow \ eroded = min((c - sediment) \cdot erodeSpeed, map[idx])$$

$$sediment > c \ \rightarrow \ deposited = (sediment - c) \cdot depositSpeed$$

Brzina i količina

```
speed = Mathf.Sqrt(Mathf.Max(0, speed * speed + deltaH *
gravity));
water *= (1 - evaporateSpeed);
if (water < 0.01f) break;
```

Za kraj moramo odrediti kolika će iznositi nova brzina čestice i koliko će od nje ostati nakon što malo ispari. Čestica ubrzava ako ide nizbrdo.

$$v_{new} = \sqrt{{v_{old}}^2 + \Delta h * gravity}$$

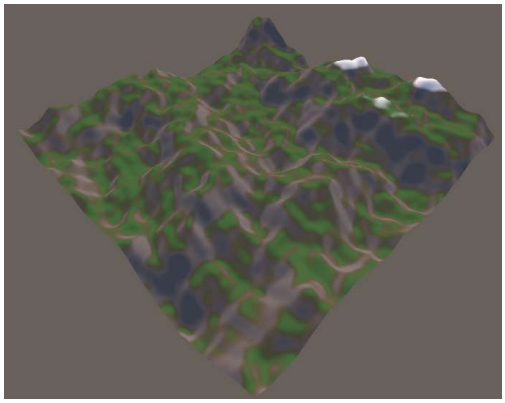
$$w_{new} = w_{old} * (1 - evaporateSpeed)$$

4.3 Parametri

U ovome poglavlju ćemo se posvetiti svim parametrima kojima korisnik ima pristup te kako ti parametri utječu na eroziju.

4.3.1 Postavke generacije terena

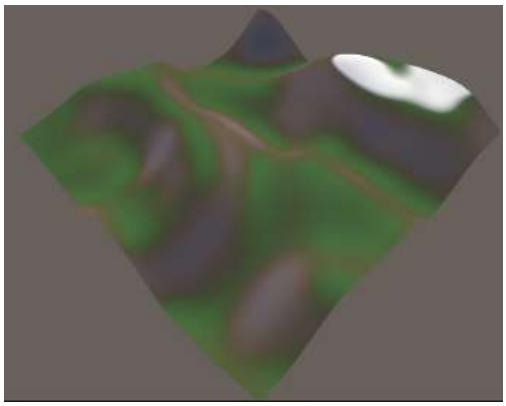
Kada generiramo teren, postavljamo mu postavke po našim željama i potrebama. Sve postavke su usko povezane uz sam algoritam Perlinovog šuma te s njima možemo baratati kako želimo. Bitno je naglasiti da su neke postavke dosta osjetljive na promjenu, pa dosta utječu promjeni izgleda terena. Pri pokretanju programa nas dočekaju inicijalne postavke za koje rezultati su dovoljno zadovoljavajući. Pri promjeni i analizi svake postavke, druge postavke ćemo ostaviti na ovoj inicijalnoj veličini. Držat ćemo Seed konstantan na 2025 kako bi bolje uočili rezultate i promjene.



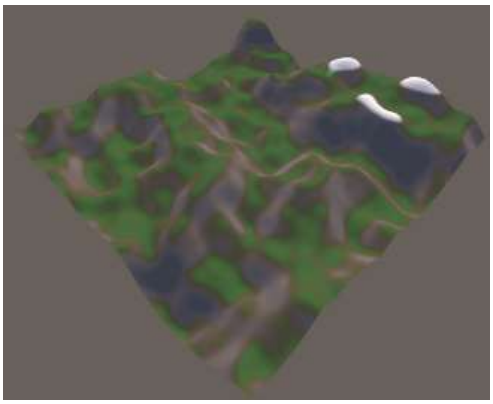
Sl. 4.2 Primjer generiranog terena sa inicijalnim postavkama, Seed = 2025

Num Octaves

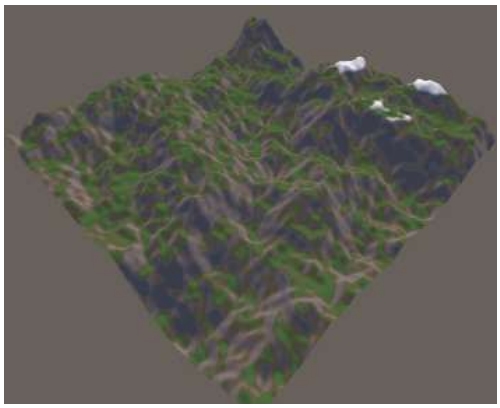
„Num octaves“ je postavka za broj oktava, to jest koliko će se puta Perlinov šum ponoviti u stvaranju slike s različitim skalama i frekvencija. Generalno što je više oktava, to je teren detaljniji.



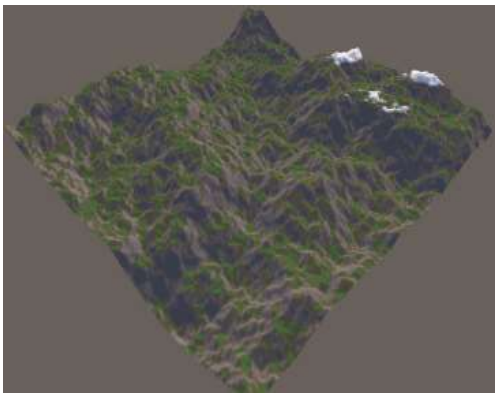
Sl. 4.3 $p_{numOctaves} = 2$



Sl. 4.4 $p_{numOctaves} = 4$



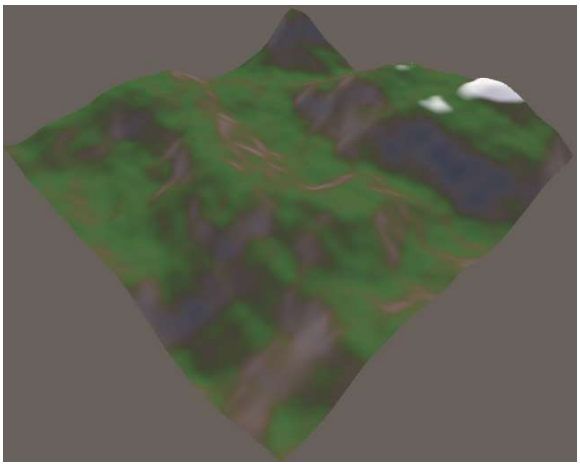
Sl. 4.5 $p_{numOctaves} = 6$



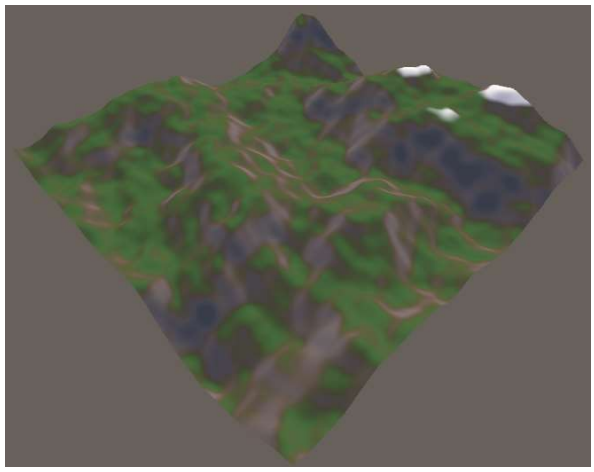
Sl. 4.6 $p_{numOctaves} = 8$

Persistance

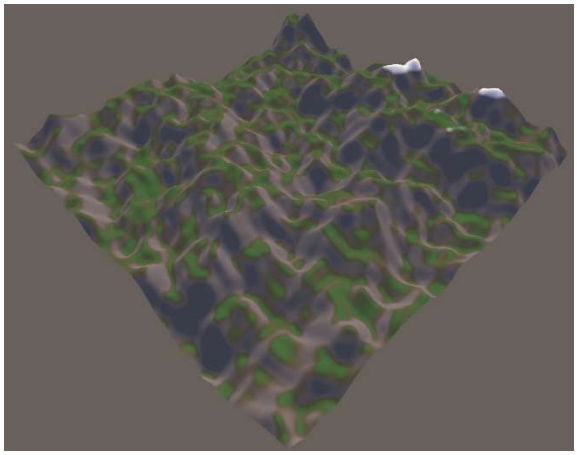
Postavka Persistance određuje koliko se doprinos svake sljedeće oktave smanjuje. To bi značilo da što je Persistance manji, svaka uzastopna vrijednost koja se generira će imati sve manji i manji utjecaj na teren.



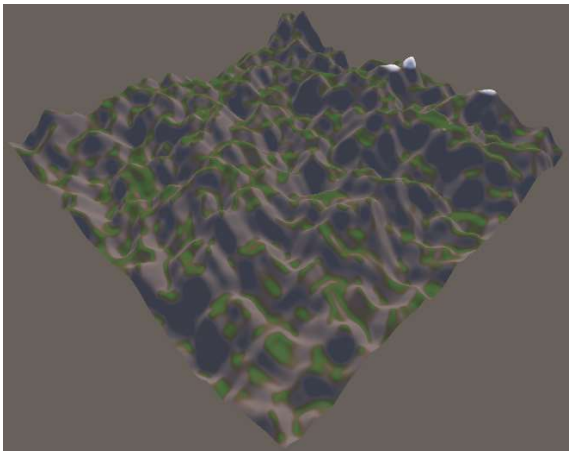
Sl. 4.7 $p_{Persistance} = 0{,}4$



Sl. 4.8 $p_{Persistance} = 0{,}5$



Sl. 4.9 $p_{Persistance} = 0{,}7$

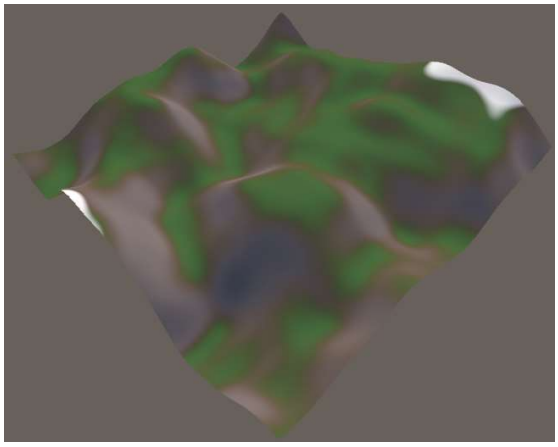


Sl. 4.10 $p_{Persistance} = 0{,}8$

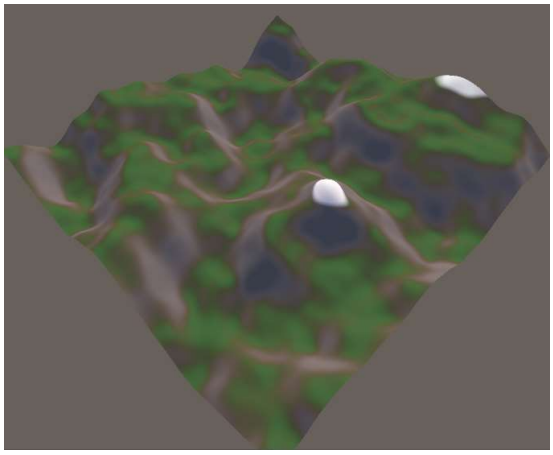
Dakle, ako je Persistance manji, svaka sljedeća oktava ima manji utjecaj na konačni teren. Vidljivo je da detalji postanu manje izraženi i teren postane glađi. No kada povećamo Persistance, više oktave postanu jače u svojem utjecaju na konačni teren.

Lacunarity

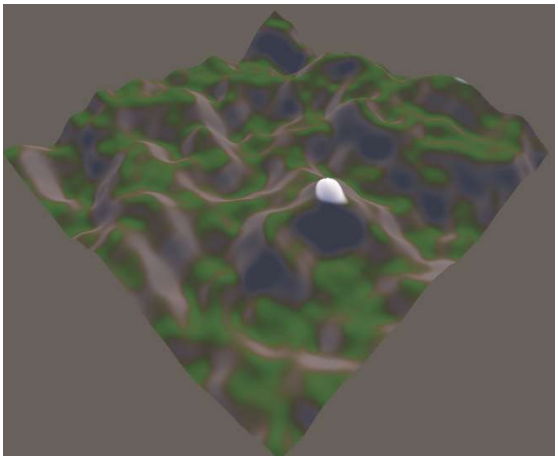
Postavka Lacunarity određuje koliko se frekvencija povećava svakom oktavom. Veća vrijednost znači da će svaka sljedeća oktava imati sitnije detalje.



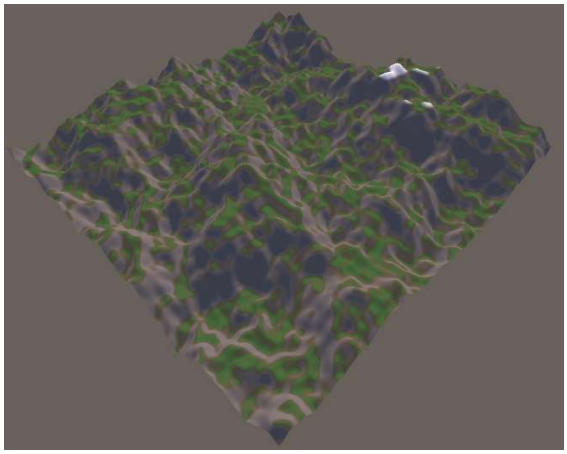
Sl. 4.11 $p_{Lacunarity} = 1{,}4$



Sl. 4.12 $p_{Lacunarity} = 1{,}6$



Sl. 4.13 $p_{Lacunarity} = 1{,}8$

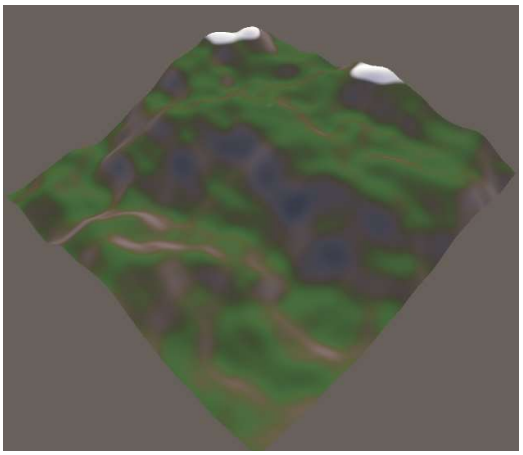


Sl. 4.14 $p_{Lacunarity} = 2$

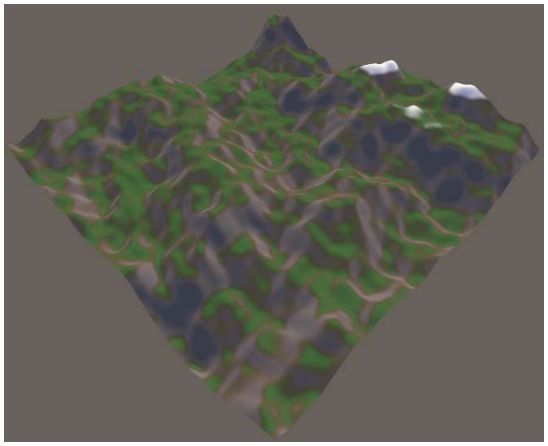
Veća vrijednost Lacunariy uzrokuje da se frekvencije svake iduće oktave mijenjaju na veću razinu. Drugim riječima, eksponencijalno utječe na koliko precizne detalje će imati svaka iduća generirana slika. S time možemo postići detaljan teren čak s niskim brojem oktava, jer prva oktava može na primjer odrediti gdje se nalaze „gore i nizine“, dok već iduća stvara detalje poput specifičnih udubina i tako dalje.

Initial Scale

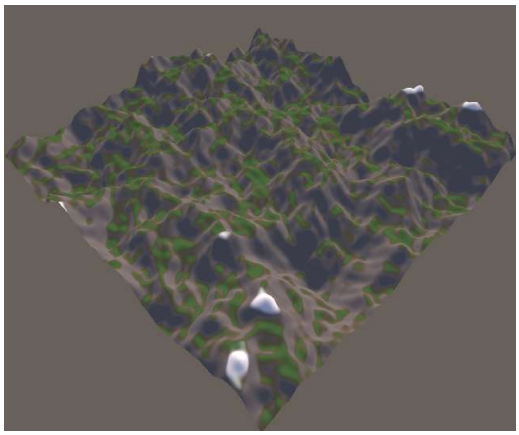
Initial scale određuje koliko je razvučen osnovni uzorak Perlinovog šuma. Manja vrijednost daje šire, glatke oblike, dok veća vrijednost daje više detalja.



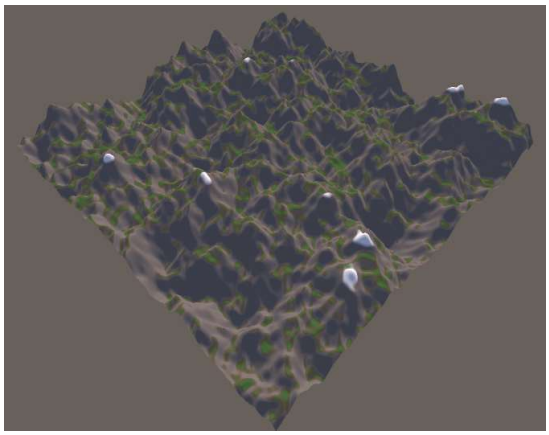
Sl. 4.15 $p_{InitialScale} = 1$



Sl. 4.16 $p_{InitialScale} = 2$



Sl. 4.17 $p_{InitialScale} = 3$

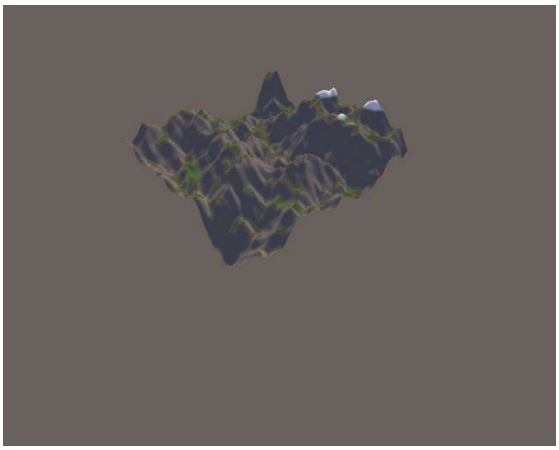


Sl. 4.18 $p_{InitialScale} = 4$

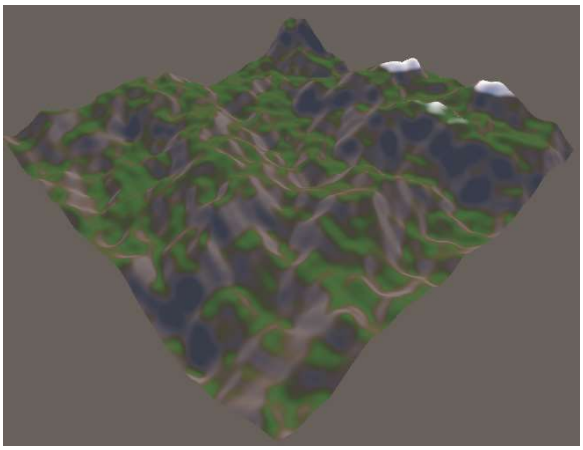
U suštini, sve što InitialScale radi je određuje koliki mali uzorak Perlinovog šuma uzimamo. Drugim riječima, koliko se približavamo, to jest udaljavamo od Perlinovog šuma kada stvaramo teren.

Map Size

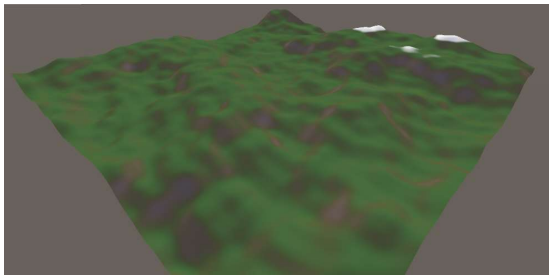
Map size određuje rezoluciju generiranog heightmapa, tj. koliko će biti piksela u x, y smjerovima terena. Primijetiti da ostale opcije ostaju iste, dakle teren postaje sve više i više ravniji jer je razvučeniji.



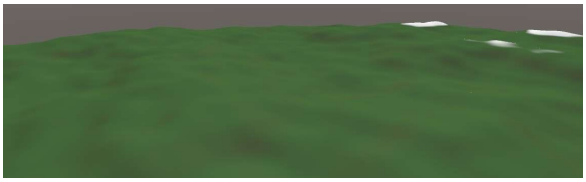
Sl. 4.19 $p_{MapSize} = 128$



Sl. 4.20 $p_{MapSize} = 256$

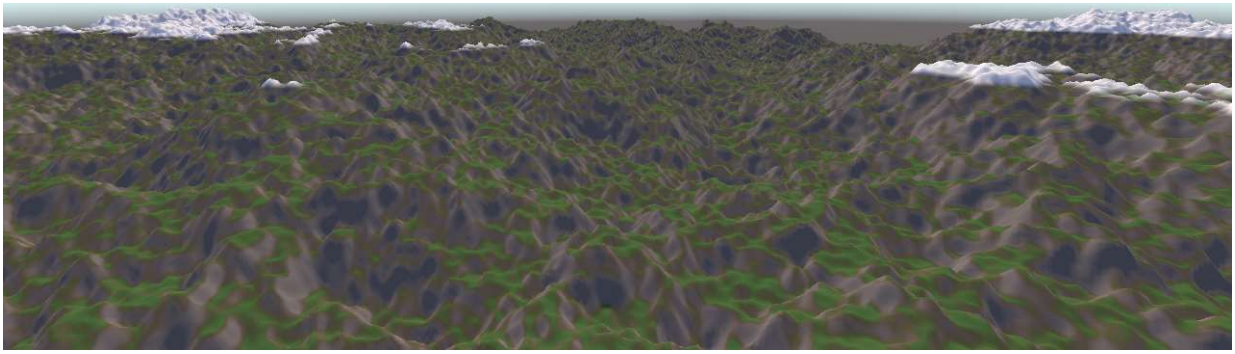


Sl. 4.21 $p_{MapSize} = 512$



Sl. 4.22 $p_{MapSize} = 1024$

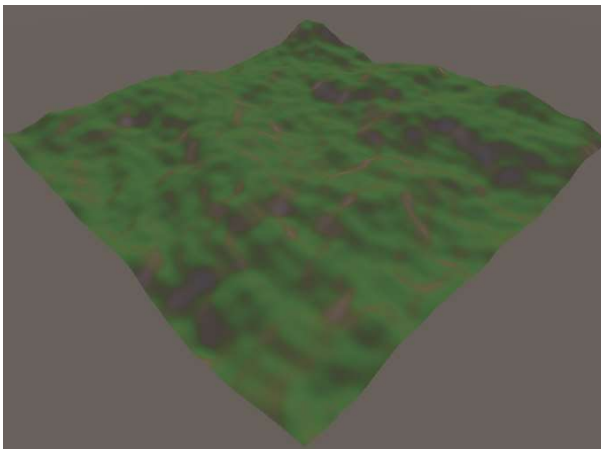
Teren iako u primjerima ne izgleda detaljan, može biti ako uzmemo u obzir ostatak opcija s kojima generiramo teren. Na primjer, malo podešavajući postavke možemo generirati ovakav teren:



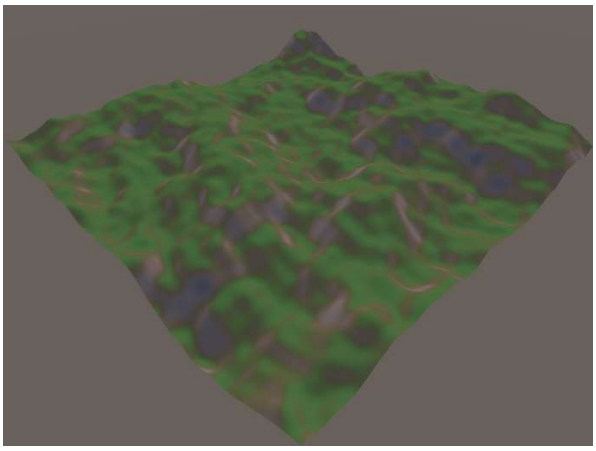
Sl. 4.23 $p_{numOctaves} = 8,\ p_{Persistance} = 0{,}65,\ p_{Lacunarity} = 1{,}95,\ p_{InitialScale} = 3,\ p_{MapSize} = 1024,\ p_{MapHeight} = 75$*)*

Map Height

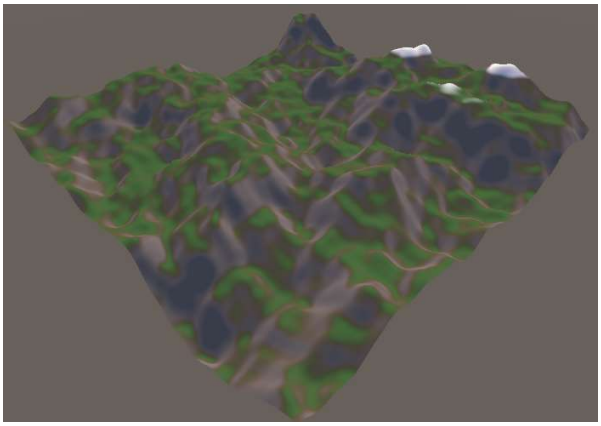
I za kraj, Map height određuje maksimalnu visinu terena. Pošto nam shader radi na konstantnoj visini, snijeg će biti iznad visine specifične visine (oko 60).



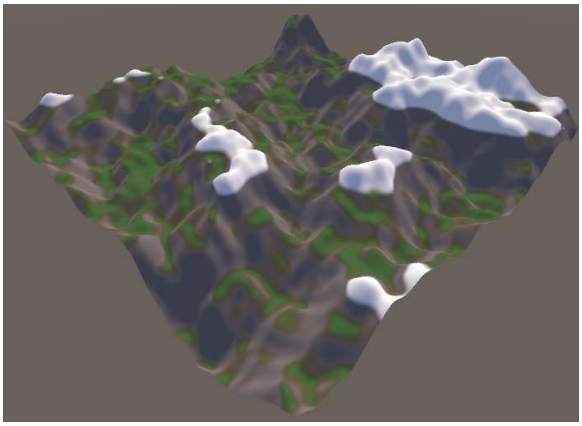
Sl. 4.24 $p_{MapHeight} = 30$



Sl. 4.25 $p_{MapHeight} = 40$



Sl. 4.26 $p_{MapHeight} = 60$



Sl. 4.27 $p_{MapHeight} = 75$

Radi na isti princip kao prethodni parametar, ali u drugoj dimenziji. Više-manje sve što radimo je razvlačimo teren u visinu.

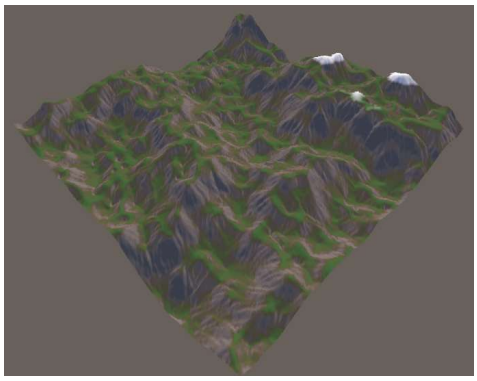
4.3.2 Postavke erozije

Inertia

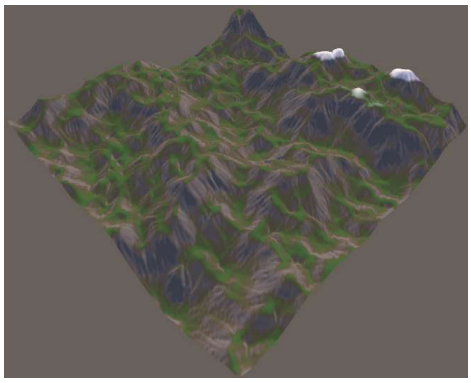
Parametar Inertia određuje koliko se čestica kreće u određenom smjeru na terenu. Sadrži vrijednosti od nule do jedinice, te se u formuli za računanje smjer kapljice množi sa smjerom kapljice i s gradijentom terena. Generalno, što je Inertia bliža jedinici, kap postaje manje diktirana sa smjerom tla i sadržava svoj smjer, dok što je bliža nuli tlo sve više i više utječe na smjer kapi. Taj fenomen se i vidi u djelu koda koji računa smjer kapi oblikovan na sljedeći način:

```
dirX = dirX * inertia - gradX * (1 - inertia)
dirY = dirY * inertia - gradY * (1 - inertia)
```

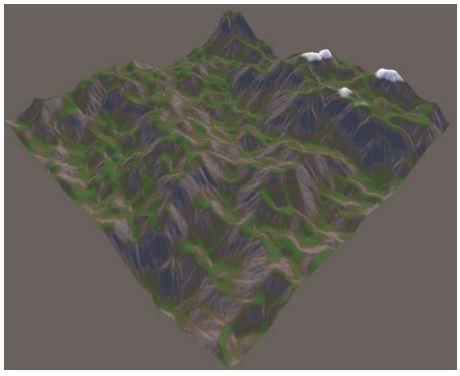
Rezultat se vidi u idućim slikama. Primijetiti da manja inercija uzrokuje dubljim jarugama (zbog toga što svaka kap koja prođe uzrokuje veću eroziju u crti kretanja, što poveća vjerojatnost da više kapi ide tim smjerom jer što je inercija manja tlo više utječe nad smjerom kretanja čestice) dok veća inercija uzrokuje nježniju eroziju.



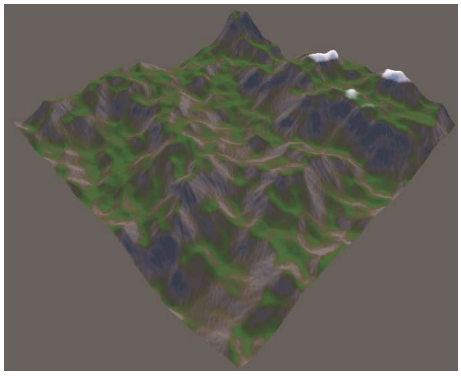
Sl. 4.28 $p_{Inertia} = 0{,}01$



Sl. 4.29 $p_{Inertia} = 0{,}25$



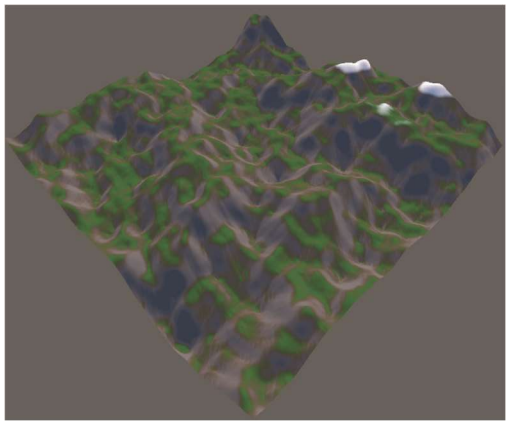
Sl. 4.30 $p_{Inertia} = 0{,}5$



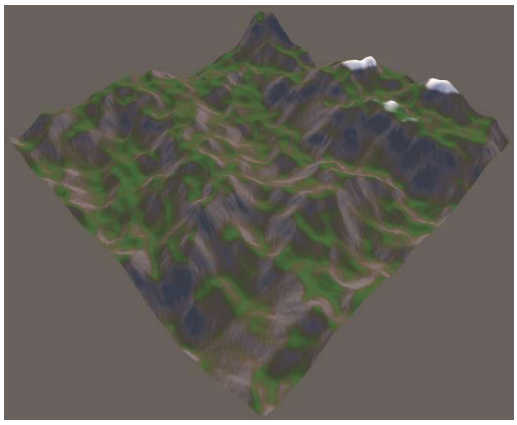
Sl. 4.31 $p_{Inertia} = 0{,}99$

SedimentCapacity

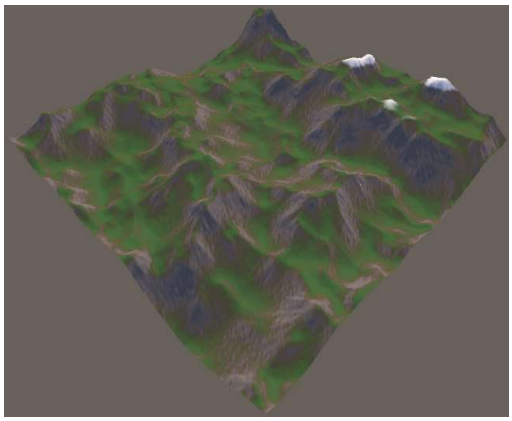
Koliko čestica ima kapaciteta nositi sedimenta zapisano je indirektno u parametru SedimentCapacity. Intuitivno, ovaj parametar direktno utječe na količinu sedimenta koja je prebačena s više točke i odložena na nekoj nižoj. U priloženim slikama vidimo očitu razliku, što je kapacitet sedimenta veći, to čestica duže prenosi sediment te ga napokon ostavlja i stvara ravne doline vidljive u primjerima ispod.



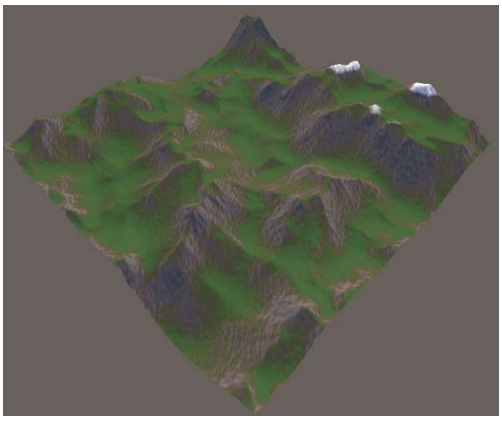
Sl. 4.32 $p_{SedimentCapacity} = 0{,}01$



Sl. 4.33 $p_{SedimentCapacity} = 0{,}5$



Sl. 4.34 $p_{SedimentCapacity} = 1$



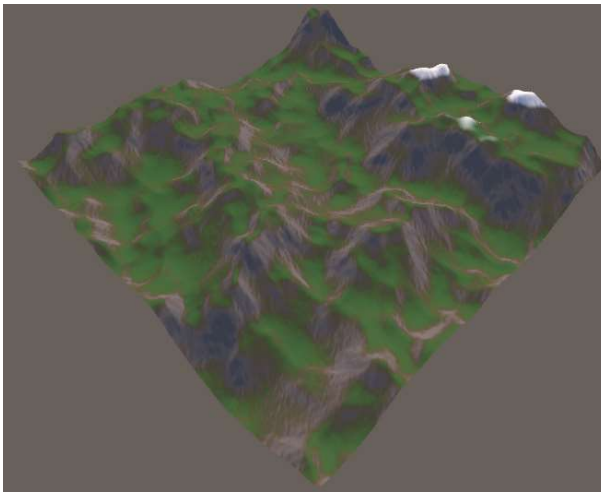
Sl. 4.35 $p_{SedimentCapacity} = 1{,}75$

Dakle, generalno veći kapacitet za sediment stvara i češću eroziju direktno zbog toga što čestica ima mogućnost uzeti na sebe teret terena češću količinu vremena. Bitno je napomenuti da ovo nije fiksna veličina već faktor kojim se množi izračun stvarnog kapaciteta, koji se određuje ovisno o faktorima količine vode, brzine i razlike u padu.

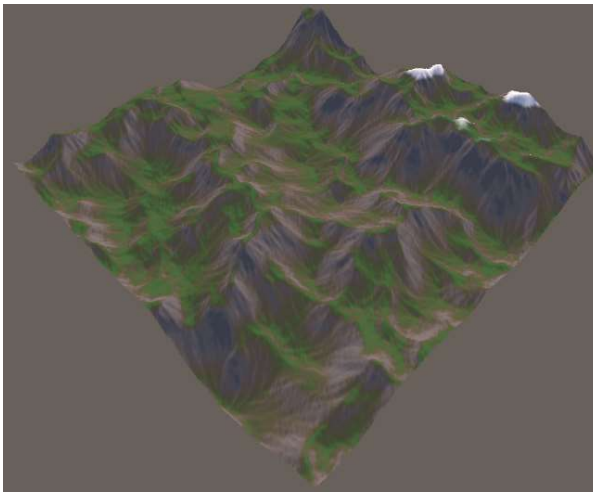
```
float capacity = Mathf.Max(-deltaH * speed * water *
sedimentCapacityFactor, minSedimentCapacity);
```

MinSedimentCapacity

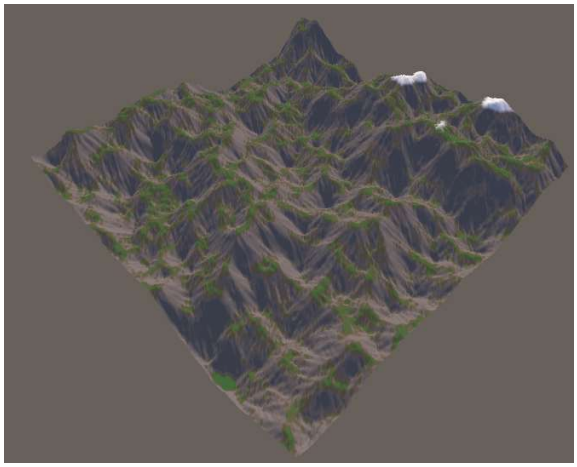
Nalik prethodnom parametru, ovaj parametar određuje koliko minimalno kapaciteta može čestica imati. Moramo imati ovaj parametar kako bi izbjegli situaciju da nam kapacitet bude nula ili još gore negativan. Dakle u gore opisanom odsječku iz koda vidimo da, ako je izračunati kapacitet ispod nule, minSedimentCapacity postaje upravo kapacitet. Povećavanjem ovoga parametra dobijemo zanimljive rezultate, u obliku vrlo grubog i neravnog terena. Koristeći intuiciju možemo zaključiti zašto je to slučaj. Kapi uzimaju sve više i više od terena i konačno umiru u najdubljim točkama, ponekad i prije nego što su dobile priliku odložiti svoj nakupljeni teren.



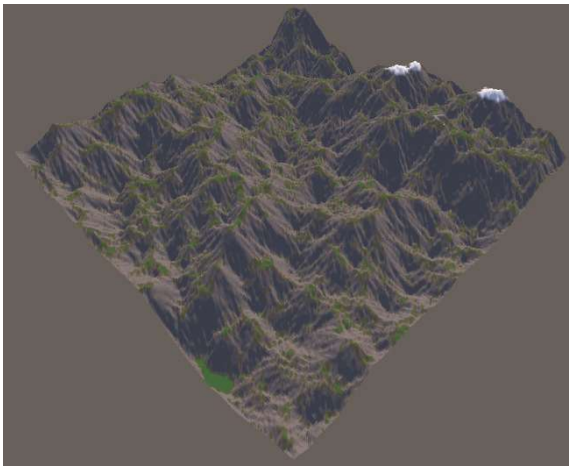
Sl. 4.36 $p_{MinSedimentCapacity} = 0{,}001$



Sl. 4.37 $p_{MinSedimentCapacity} = 0{,}01$



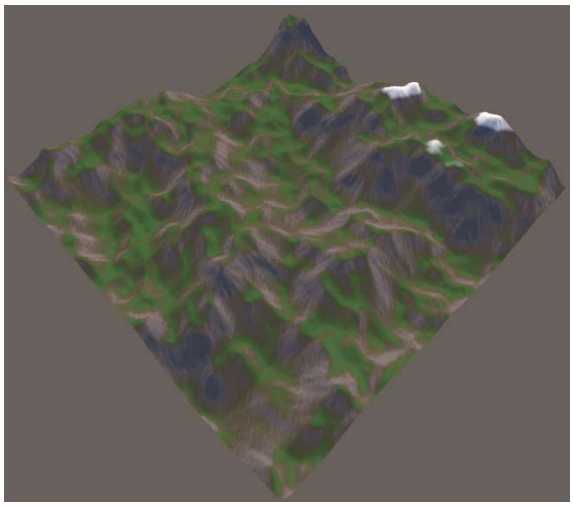
Sl. 4.38 $p_{MinSedimentCapacity} = 0{,}025$



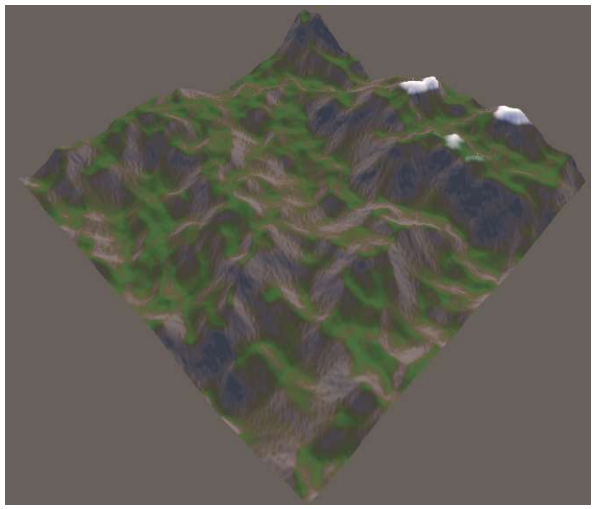
Sl. 4.39 $p_{MinSedimentCapacity} = 0{,}04$

DepositSpeed

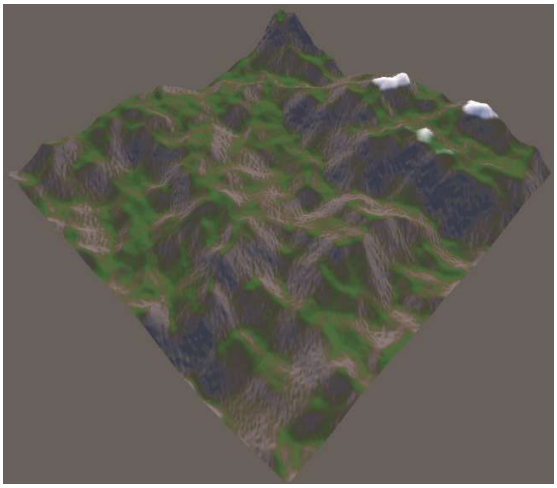
Parametar depositSpeed određuje koliko brzo čestica taloži višak sedimenta. Kako naše kapi funkcioniraju je da one, ako im je količina sedmineta veća od kapaciteta, malo po malo talože taj sediment iza sebe. Utjecaj postane jasan kada primjetimo da s većom brzinom proces je sve kraći i kraći, te teren postane hrapav upravo zbog tog razloga. Kada je proces sporiji tj. depositSpeed manji, taloženje se izduži i nastaju dulje jaruge.



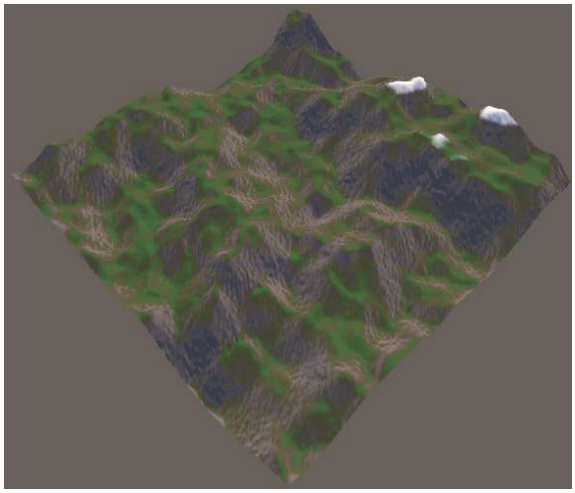
Sl. 4.40 $p_{DepositSpeed} = 1$



Sl. 4.41 $p_{DepositSpeed} = 2$



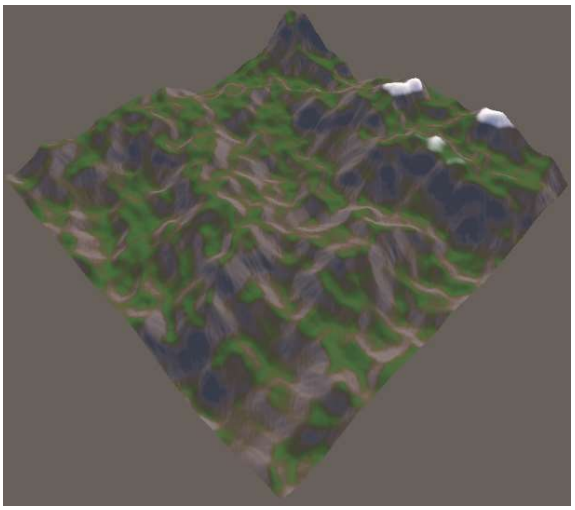
Sl. 4.42 $p_{DepositSpeed} = 4$



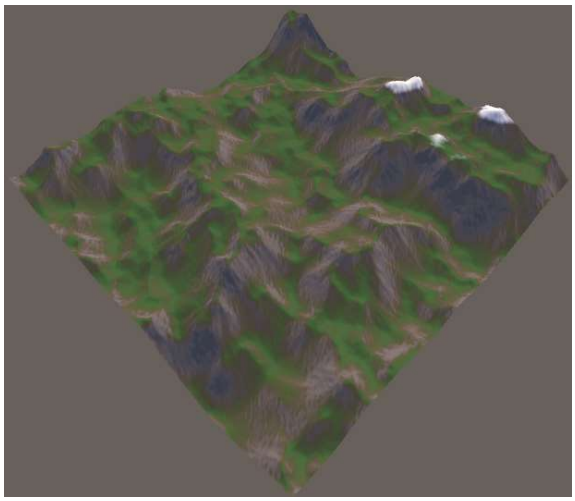
Sl. 4.43 $p_{DepositSpeed} = 6$

ErodeSpeed

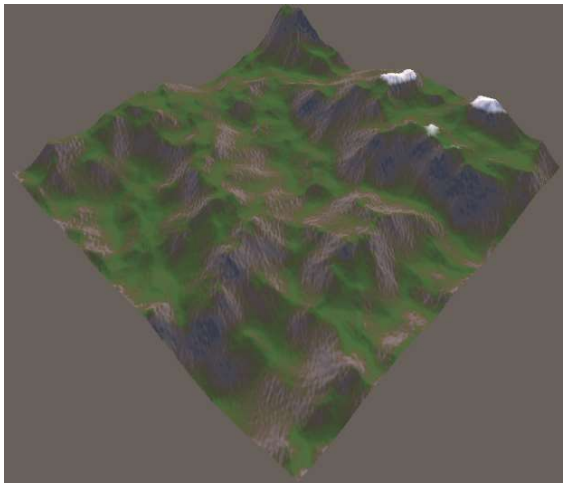
Slično depositSpeed, erodeSpeed je parametar koji određuje koliko brzo čestica uzima sediment iz terena. Što je erodeSpeed veći, kapljica je agresivnija u svojoj eroziji i stvara korodirajući efekt na teren.



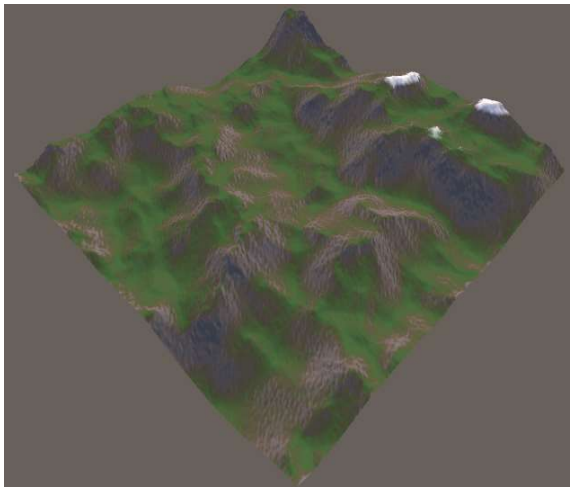
Sl. 4.44 $p_{ErodeSpeed} = 0{,}05$



Sl. 4.45 $p_{ErodeSpeed} = 0{,}2$



Sl. 4.46 $p_{ErodeSpeed} = 0{,}4$

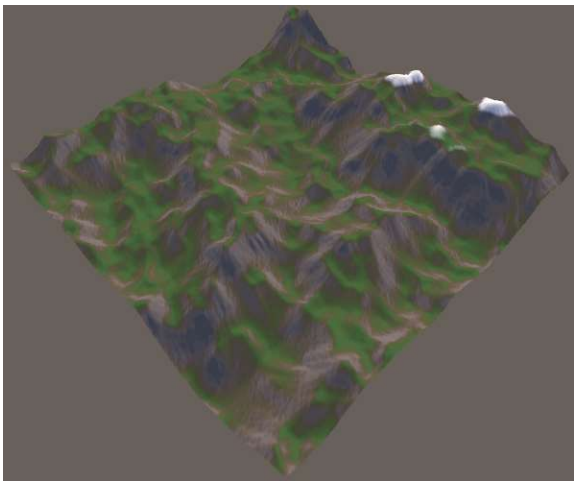


Sl. 4.47 $p_{ErodeSpeed} = 0{,}5$

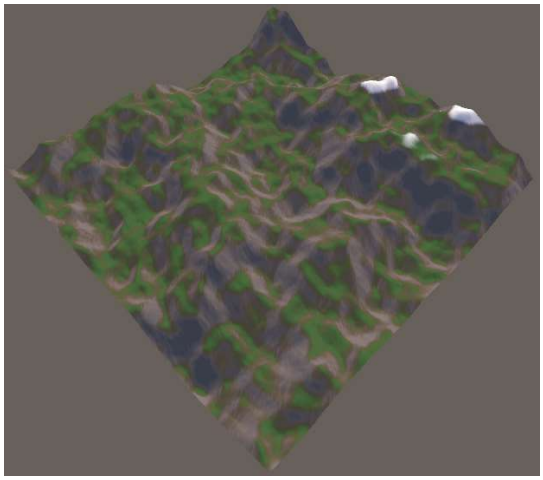
Primijetimo kako erozija utječe na teren pri velikoj brzini erodiranja. Možemo uočiti da pomalo nalikuje kiselini koja jede od predmeta i to upravo i ima smisla ako uzmemo u obzir stvarnu situaciju u kojoj bi rezultati bili slični – kisele kiše. Odličan primjer kako kisele kiše utječu na materijal su stari vrhovi zagrebačke katedrale i kako su korodirali pod utjecajem kiselih kiša.

EvaporateSpeed

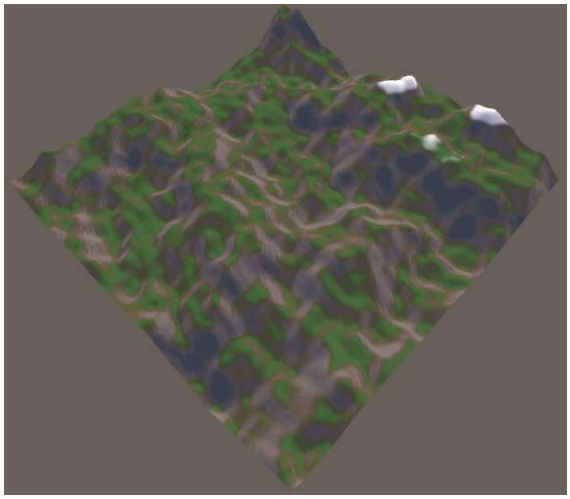
EvaporateSpeed je jednostavan parametar koji direktno utječe na brzinu isparivanja čestice. Što je EvaporateSpeed veći, to je trajanje čestice kraće, te se erozija sve manje i manje primijeti.



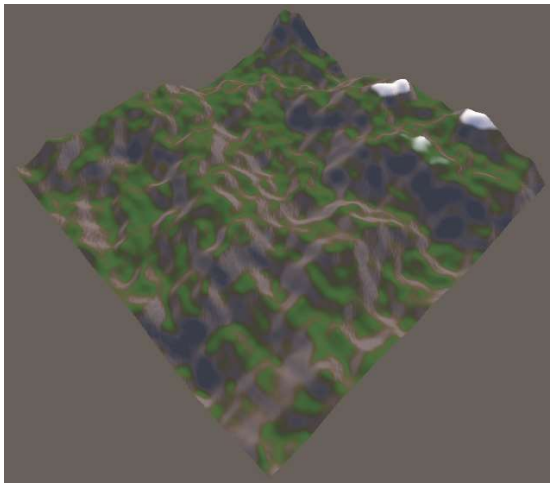
Sl. 4.48 $p_{EvaporateSpeed} = 0{,}001$



Sl. 4.49 $p_{EvaporateSpeed} = 0{,}33$



Sl. 4.50 $p_{EvaporateSpeed} = 0{,}66$



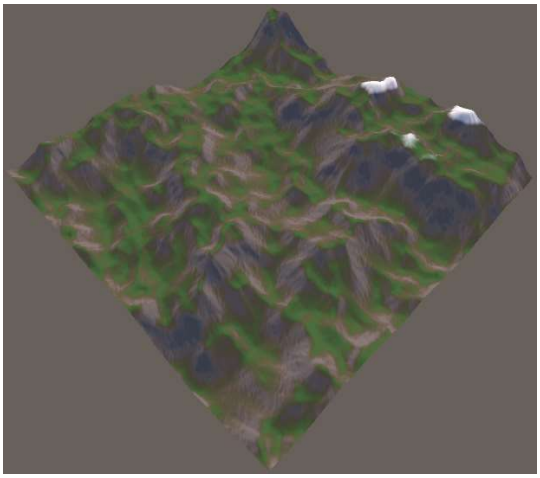
Sl. 4.51 $p_{EvaporateSpeed} = 1$

Gravity

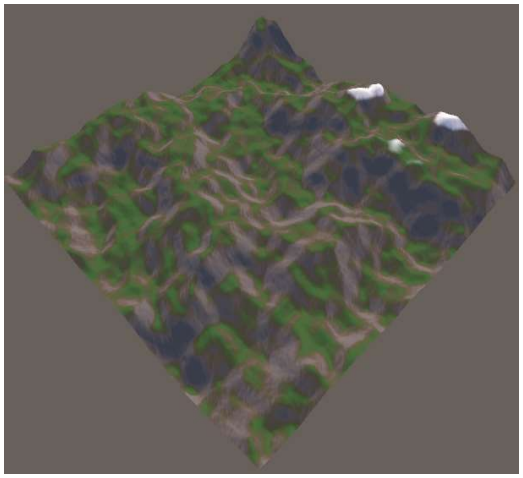
Gravity ili ti ga gravitacija je akceleracijski faktor čestice u pokretu. Bitno je naglasiti da nam gravitacija utječe na promjenu brzine te nije njezin glavni uzrok. To se primijeti kada vidimo da erozija i dalje radi unatoč činjenici da je parametar gravitacije nula, što ima smisla ako uzmemo u obzir da se čestice stvaraju s nekom početnom brzinom i one tu brzinu očuvaju tijekom svoga postojanja. Vidimo u kodu kako je to izvedeno:

```
speed = Mathf.Sqrt(Mathf.Max(0, speed * speed + deltaH *
gravity));
```

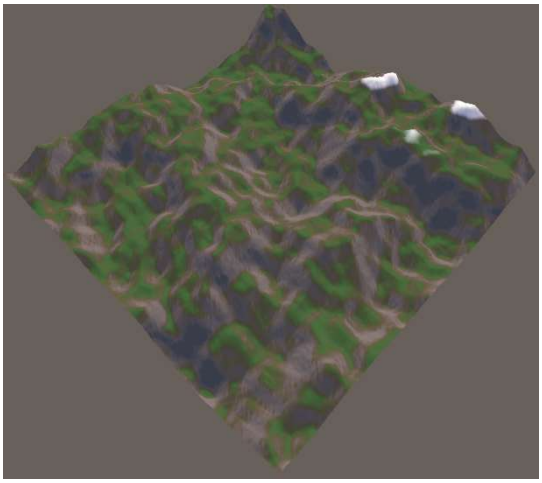
Dakle gravitacija se samo svodi na dodatni izračun nove brzine koja može biti veća ili manja, ovisno o tome idemo li uzbrdo ili nizbrdo.



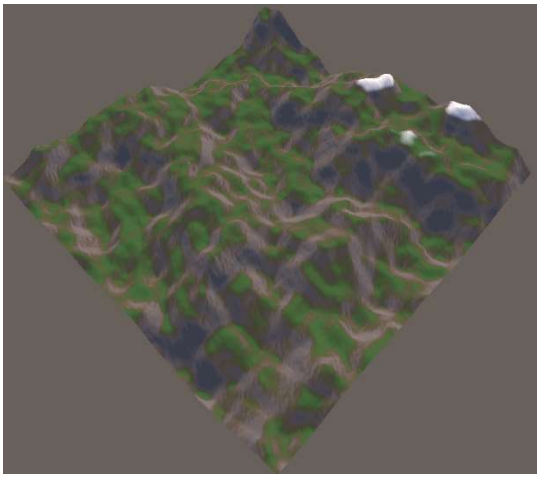
Sl. 4.52 $p_{Gravity} = 0$



Sl. 4.53 $p_{Gravity} = 25$



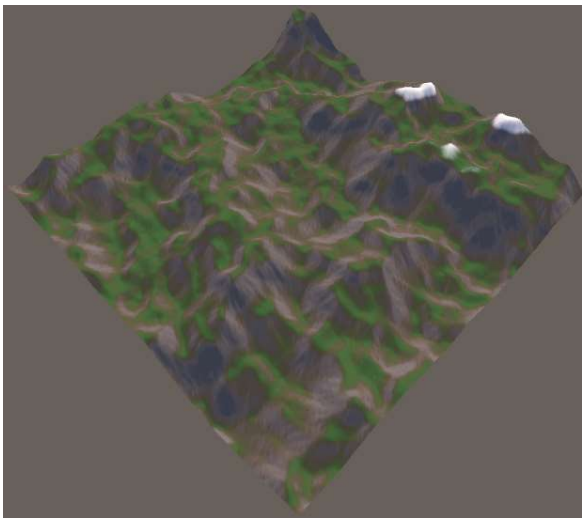
Sl. 4.54 $p_{Gravity} = 50$



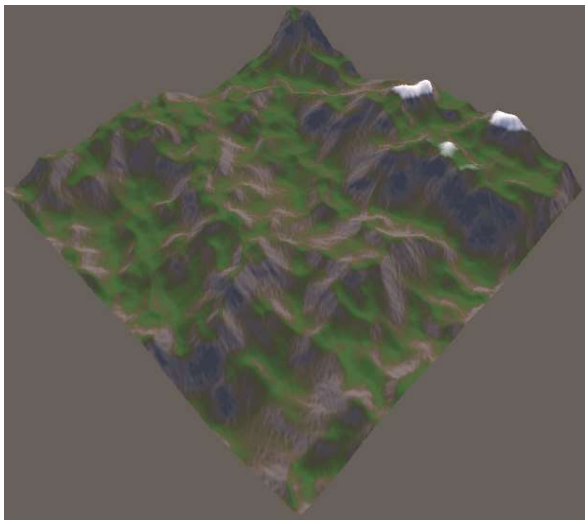
Sl. 4.55 $p_{Gravity} = 100$

WaterCount

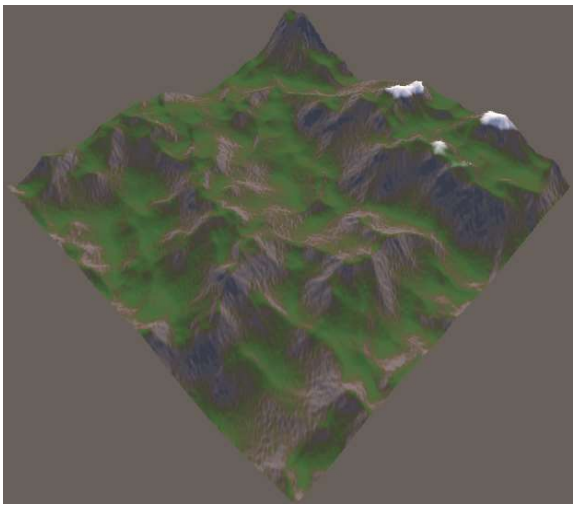
Parametar WaterCount utječe na količinu vode koja se nalazi u čestici na početku njezina stvaranja. To direktno utječe na njezin utjecaj na teren u obliku većeg kapaciteta za eroziju, dakle većom erozijom, te utječe na njezin životni vijek.



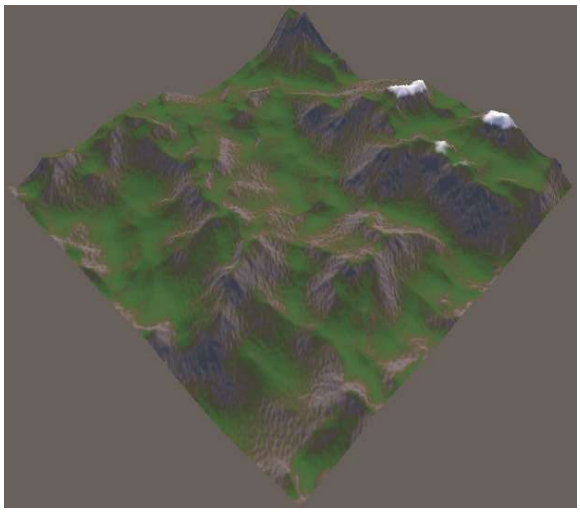
Sl. 4.56 $p_{WaterCount} = 1{,}5$



Sl. 4.57 $p_{WaterCount} = 2{,}5$



Sl. 4.58 $p_{WaterCount} = 4$

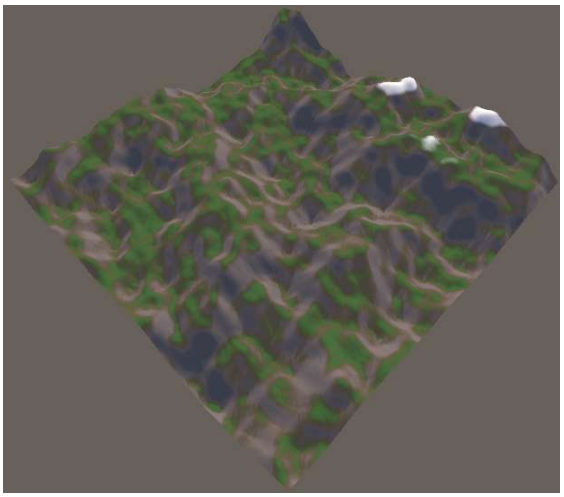


Sl. 4.59 $p_{WaterCount} = 5$

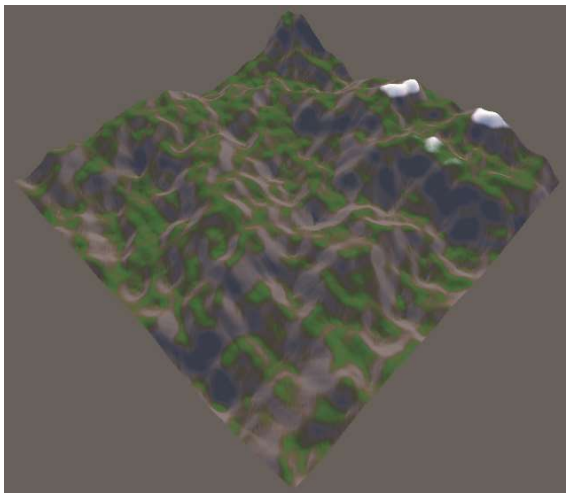
WaterCount je jedno od dva načina na koji čestica može prestat postojati. Ako je količina vode manja od fiksne veličine (u našem slučaju je ta veličina 0.01), prekidamo proces erozije za tu kap.

WaterSpeed

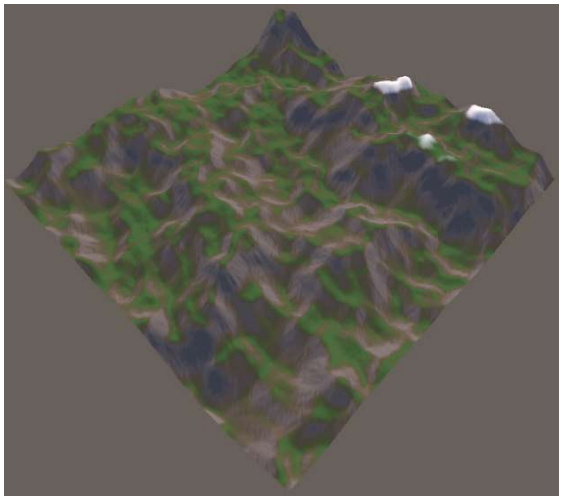
WaterSpeed određuje koliku čestica ima brzinu kada se generira na terenu. Primjetan je utjecaj ovoga parametra u slikama ispod, gdje je vidljivo da je erozija jasnija na većim nagibima u primjerima s niskom početnom brzinom. To je zbog toga što upravo utjecaj gravitacije omogući čestici da pokupi brzinu i provodi veću eroziju. Što je veća početna brzina, to kap prelazi i veću udaljenost i proces erozije se jasnije primijeti.



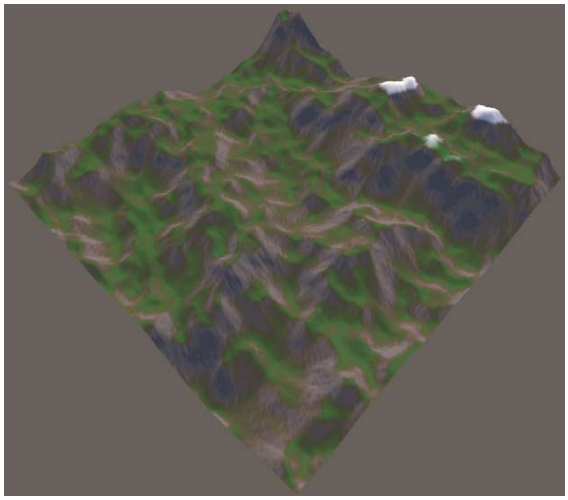
Sl. 4.60 $p_{WaterSpeed} = 0{,}01$



Sl. 4.61 $p_{WaterSpeed} = 0{,}5$



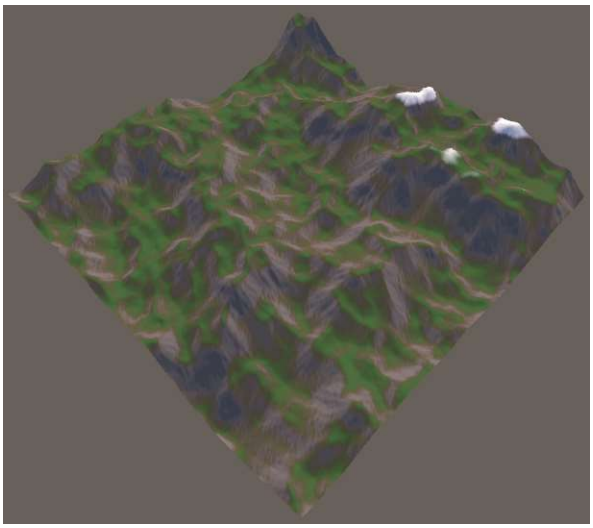
Sl. 4.62 $p_{WaterSpeed} = 1{,}5$



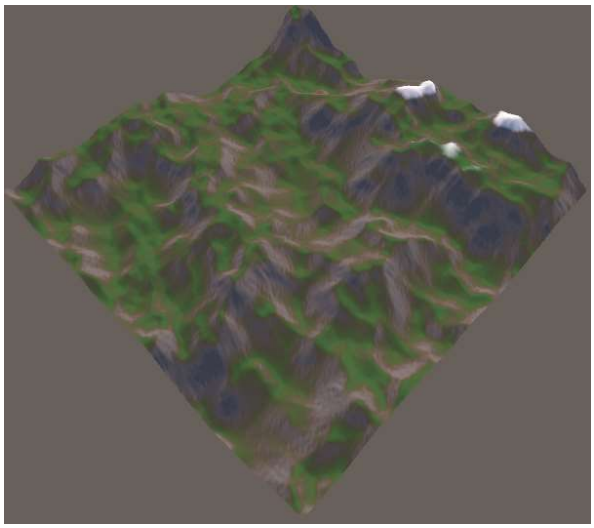
Sl. 4.63 $p_{WaterSpeed} = 2$

MaxLifetime

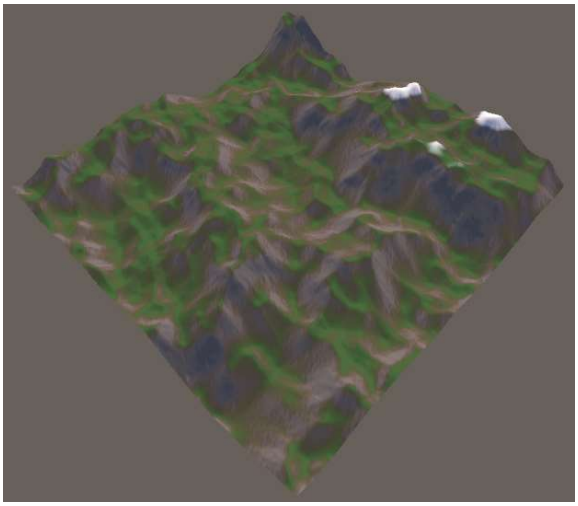
MaxLifetime je životni vijek kapljice. U interesu nam je da nemamo previše kapljica odjednom da ne moramo računati puno različitih operacija, te s time na umu imamo direktni pristup koliko dugo želimo da svaka generirana kapljica može utjecati na teren. No vidljivo je da duži vijek življenja ne uzrokuje uvijek nekakvim vidljivim rezultatima, zbog toga što imamo izrazito veliku količinu generiranih čestica, utjecaj prijašnje generiranih bude zamaskiran u tisućama drugih.



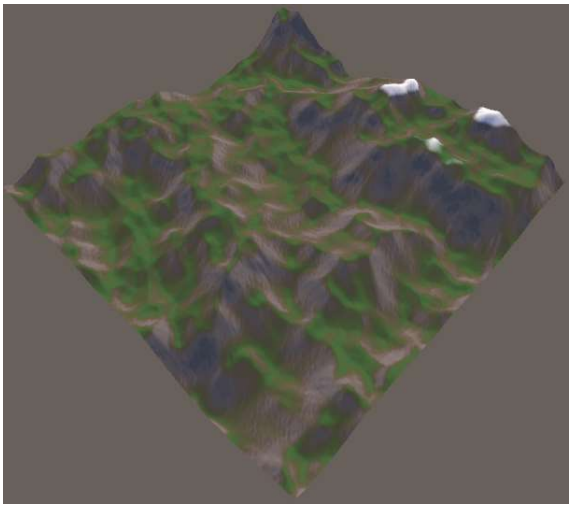
Sl. 4.64 $p_{MaxLifetime} = 25$



Sl. 4.65 $p_{MaxLifetime} = 50$



Sl. 4.66 $p_{MaxLifetime} = 75$

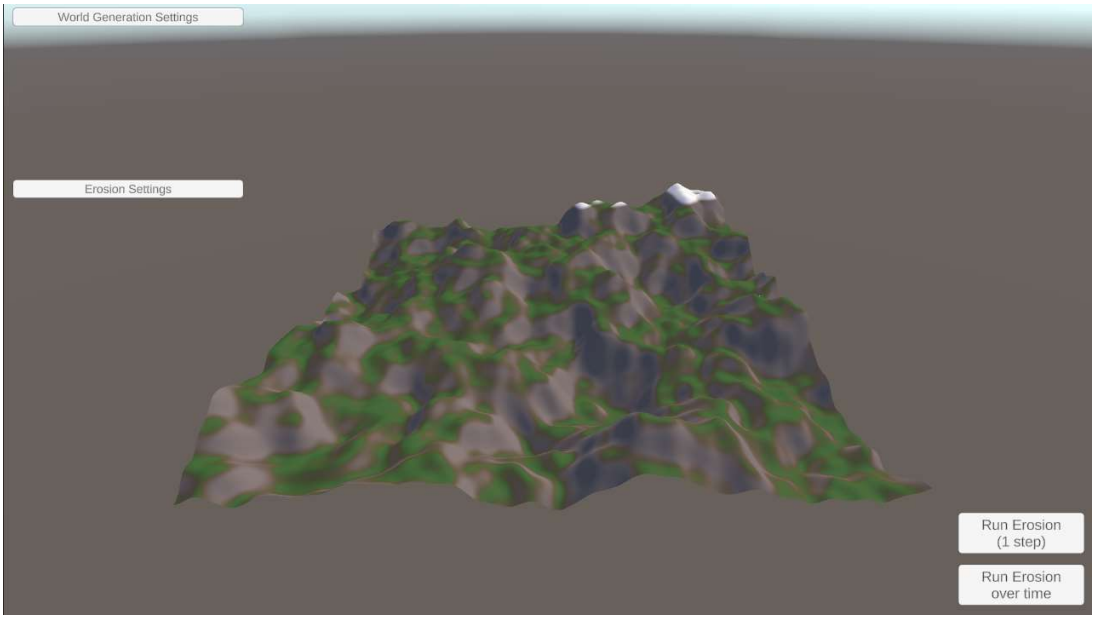


Sl. 4.67 $p_{MaxLifetime} = 100$

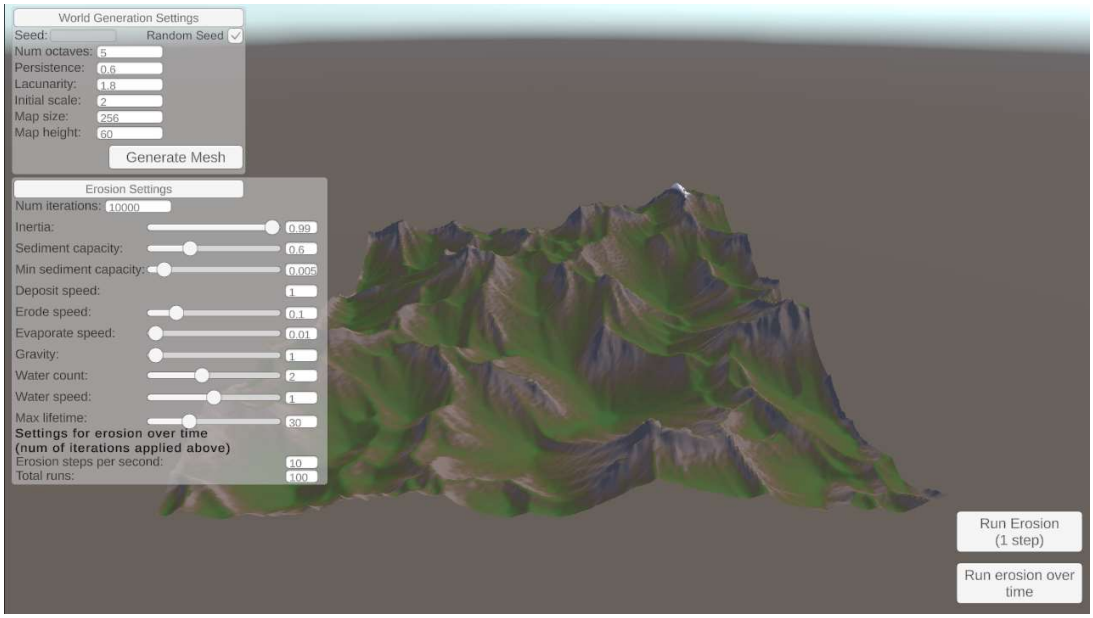
Dakle, uključujući WaterCount, ovo je drugi način na koji trenutačna čestica može prestati postojati.

4.4 Korisničko sučelje

Interakcija korisnika sa simulacijom je rađena direktno kroz alate Unity-a. Postavljamo element „Canvas“ u scenu i na njega direktno postavljamo gumbe, klizače i prozorčiće za unos precizne varijable. Sve opcije možemo sakriti gumbima za opcije da bi se mogli bolje fokusirati na teren i eroziju.



Sl. 4.68 Početni prikaz, opcije sakrivene



Sl. 4.69 Prikaz sa otkrivenim opcijama

Unity omogućuje dosta lagano spajanje skripti s korisničkim sučeljem. Dovoljno je element postaviti u scenu, to jest na Canvas i onda ga prebaciti u odgovarajuća polja u skripti. Naravno, moramo podesiti kod kako bi omogućio unos preko tih elemenata, i omogućio gumbe da provode akcije. Također je bitno varijable koje želimo promijeniti postaviti na javne kako bi korisnik imao pristup promjenama tih varijabli. Bitno je naglasiti da se decimalni zarez koristi za unos varijabli s decimalnim djelom. Uz sve to korisnik i može pokretati se oko terena kako želi i detaljnije vidjeti koji ga dijelovi zanimaju.

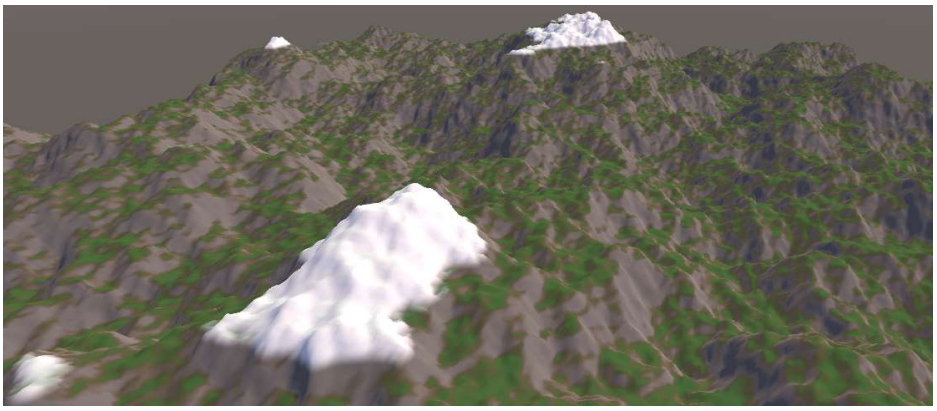
5. Rezultati

S definiranim svim definiranim formulama preslikanim onima iz stvarnoga svijeta, parametrima kojima dobivamo pristup radu simulatora i samog sučelja, ostvarujemo konačan produkt kao računalna simulacija erozije terena. Sve elemente spajamo i uočimo plod stvaralaštva, te sada možemo prikazati različite scenarije, prikaze i kolektivan izlaz simulacije.

U poglavlju razjašnjenja parametara prikazan je utjecaj svih parametara na konačan rezultat stvaranja terena, odnosno postavka erozije. No fiksiranjem na jedan parametar nas onemogućuje da uočimo svestranost simulacije. S time na umu, osvrnuti ćemo se na nekolicini primjera nastalih kombiniranjem različitih postavki za generiranje terena te onda kombiniranjem postavki za erodiranje istog. S time možemo vidjeti kako i što simulacija može, no i prikazati njezine granice.

Generiramo planinski lanac sa sljedećim postavkama:

$$p_{numSeed} = 1234,\ p_{numOctaves} = 7,\ p_{Persistance} = 0{,}45,\ p_{Lacunarity} = 2{,}4,$$
$$p_{InitialScale} = 3,\ p_{MapSize} = 512,\ p_{MapHeight} = 70$$



Sl. 5.1 Generirani planinski lanac

Prikazat ćemo najprije slučaj gdje je erozija gruba, ostavlja velike jaruge i oštro reže teren. Htjeli bi rezultate slične onima u stvarnome svijetu, u Dolini smrti u Kaliforniji, SAD.

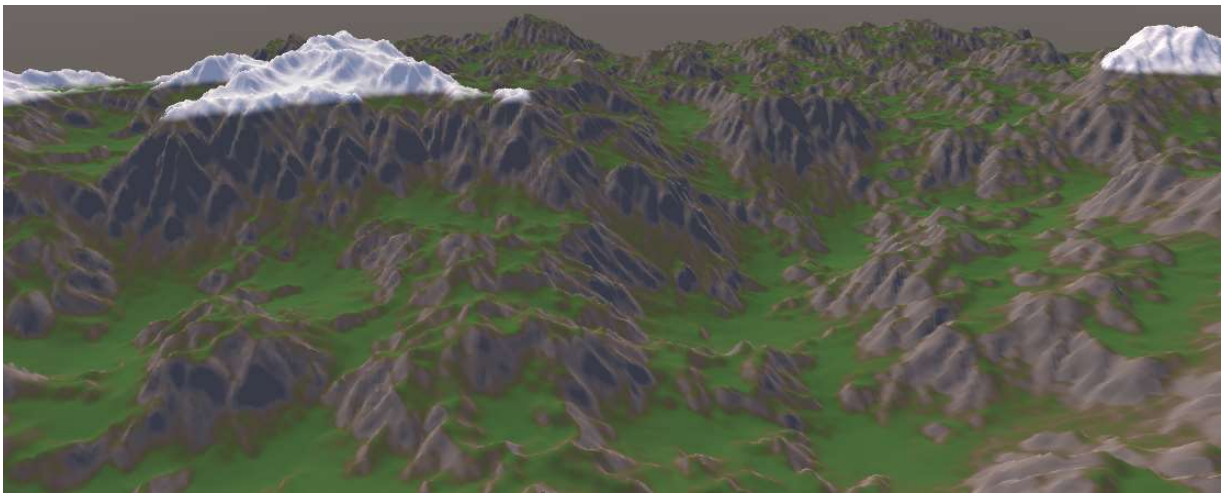


Sl. 5.2 Dolina smrti, preuzeto sa National Audubon Society

Postavimo postavke kao sljedeće bitno je staviti inerciju na nisku:

$$p_{Inertia} = 0{,}005, \quad p_{SedimentCapacity} = 0{,}6, \quad p_{MinSedimentCapacity} = 0{,}001,$$
$$p_{DepositSpeed} = 1, \quad p_{ErodeSpeed} = 0{,}1, \quad p_{EvaporateSpeed} = 0{,}01,$$
$$p_{Gravity} = 1, \quad p_{WaterCount} = 2, \quad p_{WaterSpeed} = 1, \quad p_{MaxLifetime} = 30$$

Te dobijemo sljedeće rezultate, s inspiracijom Doline Smrti:



Sl. 5.3 Teren nalik Dolini smrti

Jaruge su izražene i doline su specifične i veoma ravne, zbog visoke količine sedimenta. Ovo bi bio primjer brze erozije i slabog materijala (često kamena vapnenca i sličnog)

S time prikazanim, probat ćemo stvoriti drugačiji primjer. Okrenimo se prema planini Biokovo i njezinim karakterističnim izgledom.

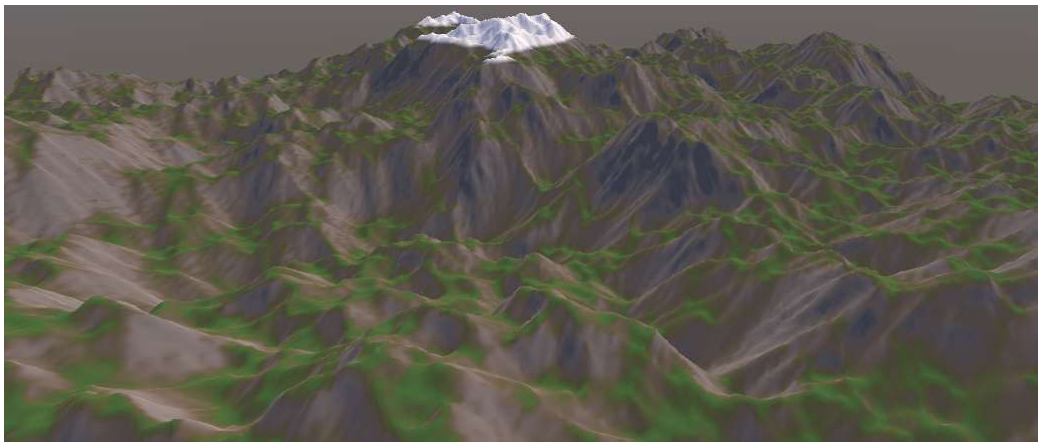


Sl. 5.4 Biokovo, preuzeto sa Luxuria

Biokovo je primjer masiva s mnogo ponikva, jama i zelenila, takozvani mrežasti krš. Pokušajmo replicirati izgled u našoj simulaciji.

Postavimo parametre kao sljedeće:

$$p_{Inertia} = 0{,}99, \quad p_{SedimentCapacity} = 0{,}7, \quad p_{MinSedimentCapacity} = 0{,}008,$$
$$p_{DepositSpeed} = 2, \quad p_{ErodeSpeed} = 0{,}1, \quad p_{EvaporateSpeed} = 0{,}01,$$
$$p_{Gravity} = 3, \quad p_{WaterCount} = 2, \quad p_{WaterSpeed} = 1, \quad p_{MaxLifetime} = 30$$



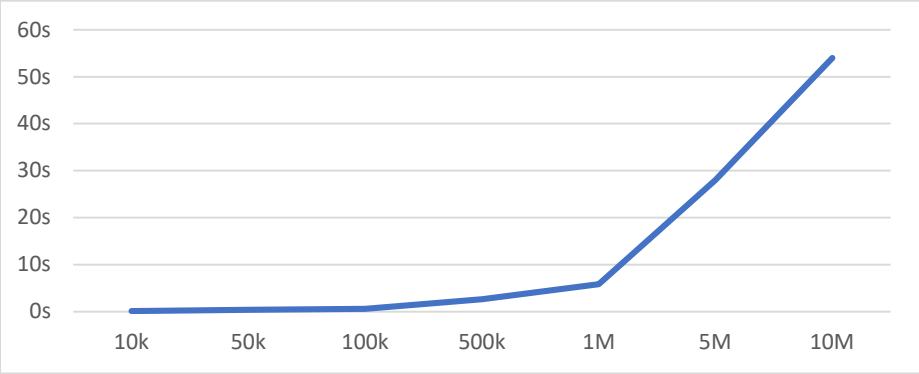
Sl. 5.5 Teren nalik Biokovu

Cilj prikaza ova dva primjera je uspostavljanje da erozija nije ista u svim krajevima svijeta. Puno su faktora u igri, te s našom implementacijom možemo samo približno dobiti rezultate stvarnoga svijeta. Primjeri poput različitog faktora erozije različitih materijala, različita pH

vrijednost kiše u krajevima svijeta, vjetrovi, zasićenost korijena drveća koje stvaraju otpor i pružaju podršku tlu te mnogi drugi čimbenici utječu na kako erozija oblikuje svijet oko nas.

Vidjevši rezultate, možemo se osvrnuti na postavljeni cilj simulacije i obuhvatiti njegovu ispunjenost. Potrebno je bilo napraviti algoritam za stvaranje erozije na terenu s mogućnostima konfiguracije. Primjećujemo da je efekt erozije prikazan podosta realističan uz očekivana ograničena korištenih alata, specifično Unity. No bitno je naglasiti da sama logika algoritma, način uporabe i implementacije je univerzalan i moguće je koristiti istu logiku u drugom okruženju, bili to pogonski sklop neke igre ili realističniji prikaz u obliku simulacije fizike. U tom smislu, sami zahtjevi rada su ispunjeni bili u ostvarivanju ovoga cilja, uz dodatnu mogućnost generiranja terena uporabom Perlinovog šuma.

Sa svime time na umu i konkretnim primjerima iz svijeta, rezultati postaju zadovoljavajući. Uz to još, zbog jednostavnosti nekih računa te načina ušteda računalne snage, simulacija je veoma brza u svome izračunu i ima mogućnost nježnog prikaza erozije. Većinski jedine granice prikazane u simulaciji dolaze od korištenja Unity-a, iako je poslužio više nego što je limitirao. Zbog jednostavnosti korištenja i efikasnosti rješenja, sama realizacija simulacije nije bila problem. U nastavku dodatno prilažemo tablicu vremena potrebnih za generiranje erozije u prethodno opisanom scenariju Biokovo, s prije navedenim parametrima:



Graf 1 Vrijeme izvođenja (y) za broj iteracija (x)

Primijetimo da simulacija konvergira oko 10M iteracija na mome računalu. (osam jezgri, 4.4 GHz procesor)

Zaključak

Cilj završnoga rada je ostvarivanje erozije, te sam probao na taj cilj postići sve što mogu u svim elementima oko njega kako bi imao bolji prikaz samoga procesa. Napravio sam nasumično generiran teren, jednostavan sjenčar za prikaz vegetacije, kamena i sličnog, te interaktivnu simulaciju kako bi i korisnik mogao proučiti malo parametre algoritma. Sve to s konačnim ciljem da se prikaže ta stvarna pojava erozije koja se konstantno dešava oko nas, a da mi to ne primijetimo.

Pristupajući završnome radu, vizija mi je bila jasna, no ubrzo sam naišao na skupinu problema koje su me zakočile. Pokušao sam razumjeti koncepte gradijenta, lokalnih minimuma i računanja puta nasamo, no ubrzo sam shvatio da je problem kompliciraniji nego što sam se njemu nadao. Zbog toga, gledajući literaturu naišao sam na dosta radova koji se bave istom temom kao i tema ovoga rada, te koji su prikazali rješenja problema zbog koji su me kočili. Specifično najveći problem koji me držao u početku pisanja algoritma je rješavanje lokalnih minimuma i koncepta inercije. Pokušavao sam konstantno da mi kapi ne ulijeću u prvu jamu koju naiđu te sam ubrzo shvatio da nisam jedini naišao na takav problem, te sam rješenje tek našao u radu H. Bayera [4].

Za sam algoritam postoje mogućnosti nadogradnje s ciljem boljih i realističnijih rezultata. U smislu same izvedbe, potencijalno bi poboljšalo performanse korištenje jezika HLSL da bi omogućili grafičkoj kartici izračun sjenčanja i poboljšali vizualni prikaz simulacije. Također, mogli bi izvedbu probati u drugim okružjima ili probati sami napisati pogonski sklop za fiziku u višim jezicima poput C++. Dodatne stavke koje mogu služiti kao moguće nadogradnje trenutačnome algoritmu su mogućnost više vrsta materijala, da se svaki materijal s različitim faktorom erodira. To bi promijenilo eroziju na područjima s dosta kamenja, gdje bi većina kamena sporije erodirala od na primjer zemlje taložene na njemu. Ovakva promjena bi omogućila i stvaranje kanjona i kanala. Dodatna moguća nadogradnja je implementacija ubacivanja vanjske visinske mape za prikaz erozije u nekom već poznatom scenariju.

No unatoč mogućim nadogradnjama, samom završnom radu sam pristupio s ciljem u glavi, te taj cilj mi je i više nego ispunjen. Prikaz erozije je omogućen kroz jednostavnu simulaciju, te intuitivno imamo mogućnost interakcije i detaljnog pregleda terena.

Literatura

[1] J. Gilley, *Water Erosion* 2005. Poveznica: https://www.sciencedirect.com/topics/earth-and-planetary-sciences/water-erosion. Pristupljeno: 16. lipnja 2025.

[2] F. R. Baneš. B, *Visual simulation of hydraulic erosion* 2002. Poveznica: https://www.researchgate.net/publication/221546395_FORSBACH_R_Visual_simulation_of_hydraulic_erosion. Pristupljeno: 26. lipnja 2025.

[3] V. T. J. H. S. K. B. B. Baneš, *Hydraulic erosion* Izdanje: Wiley InterScience, West Lafayette, 2005.

[4] H. T. Beyer, *Implementation of a method for hydraulic* TECHNISCHE UNIVERSITÄT MÜNCHEN, 2015.

[5] J. Nett, *Bilinear Interpolation* Poveznica: https://web.pdx.edu/~jduh/courses/geog493f09/Students/W6_Bilinear%20Interpolation.pdf. Pristupljeno: 16. lipnja 2025

[6] A. C. N. Mostoufi, ScienceDirect 2023. Poveznica: https://www.sciencedirect.com/topics/computer-science/central-difference. Pristupljeno: 16. lipnja 2025

[7] Unity Technologies, *Unity 6.1 User Manual*, Poveznica: https://docs.unity3d.com/Manual/index.html. Pristupljeno: 27. lipnja 2025.

[8] Wikipedia, »Perlin Noise,« Poveznica: https://en.wikipedia.org/wiki/Perlin_noise. Pristupljeno: 18. lipnja 2025

Sažetak

Simulacija erozije terena uzrokovane vodom

Računalno rješenje simulacije erozije terena uzrokovane vodom. Na nasumično generiran teren djeluje veliki broj čestica vode koje svojim kretanjem mijenjaju njegov izgled, stvarajući efekt erozije. S pomoću algoritma za eroziju ostvarena su fizikalna pravila iz stvarnoga svijeta u računalne instrukcije koje simuliraju dugi vremenski proces erozije u ljudski vidljivo vrijeme. Korisnik ima mogućnost mijenjati parametre erozije i generacije terena preko korisničkog sučelja.

Ključne riječi: erozija terena, visinska mapa, simulacija

Summary

Simulation of terrain erosion caused by water

A computer-based solution for simulating water-induced terrain erosion. A randomly generated terrain is subjected to a large number of water particles that change its appearance with their movement, creating an erosion effect. Using an erosion algorithm, the real-world physical rules are translated to computer instructions that simulate the long-term process of erosion in human-perceivable time. The user can change erosion and terrain generation parameters through the user interface.

Key words: terrain erosion, heightmap, simulation

Skraćenice

HLSL	*High-Level Shader Language*	*Visoko-razinski jezik za sjenčanje*